

POLITECNICO
MILANO 1863

COMPUTER SCIENCE AND ENGINEERING

SOFTWARE ENGINEERING II

2025 – 2026

RASD

Requirement Analysis and Specification Document

Best Bike Paths (BBP)

Andrea Rubagotti [286446], Maria Concetta Santagata [306018], Samuele Leanza [991114]

December 23, 2025

Version 1.0

Contents

1 INTRODUCTION **3**
- 1.1 Purpose . . . 3
  - 1.1.1 Goals . . . 3
- 1.2 Scope . . . 3
  - 1.2.1 World Phenomena . . . 4
  - 1.2.2 Shared Phenomena . . . 4
  - 1.2.3 Machine Phenomena . . . 5
- 1.3 Definitions, Acronyms, Abbreviations . . . 5
  - 1.3.1 Definitions . . . 5
  - 1.3.2 Acronyms . . . 5
  - 1.3.3 Abbreviations . . . 5
- 1.4 Revision History . . . 6
- 1.5 Document Structure . . . 6

2 OVERALL DESCRIPTION **6**
- 2.1 Product perspective . . . 6
  - 2.1.1 Scenarios . . . 6
  - 2.1.2 Domain class diagram . . . 8
  - 2.1.3 State diagrams . . . 9
- 2.2 Product functions . . . 10
  - 2.2.1 Create account and Login . . . 10
  - 2.2.2 Manual insertion of Bike Paths Data . . . 10
  - 2.2.3 Automatic Data Acquisition . . . 10
  - 2.2.4 Bike Paths visualization on Map . . . 11
  - 2.2.5 Score Calculus . . . 11
  - 2.2.6 Paths information data filtering, cleaning, merging . . . 11
- 2.3 User characteristics . . . 11
  - 2.3.1 Registered User . . . 11
  - 2.3.2 Non-Registered User . . . 12
- 2.4 Assumptions, dependencies and constraints . . . 12
  - 2.4.1 Regulatory policies . . . 12
  - 2.4.2 Domain Assumptions . . . 12

3 SPECIFIC REQUIREMENTS **14**
- 3.1 External Interface Requirements . . . 14
  - 3.1.1 User Interfaces . . . 14
  - 3.1.2 Hardware Interfaces . . . 19
  - 3.1.3 Software Interfaces . . . 19
  - 3.1.4 Communication Interfaces . . . 20
- 3.2 Functional requirements . . . 20
  - 3.2.1 Use Case Diagram . . . 22
  - 3.2.2 Use Cases . . . 24
  - 3.2.3 Sequence Diagrams . . . 32
  - 3.2.4 Requirement Mapping . . . 38
- 3.3 Performance Requirements . . . 38
- 3.4 Design Constraints . . . 39
  - 3.4.1 Standards compliance . . . 39
  - 3.4.2 Hardware limitations . . . 39
  - 3.4.3 Any other constraint . . . 39

3.5 Software System Attributes 39
3.5.1 Reliability 39
3.5.2 Availability 40
3.5.3 Security 40
3.5.4 Maintainability 40
3.5.5 Portability 40

4 FORMAL ANALYSIS USING ALLOY 41

5 EFFORT SPENT 66

6 REFERENCES 66

1 INTRODUCTION

1.1 Purpose

In recent years, awareness of personal health has significantly grown and people pay attention to practise more physical activity. From the huge variety of existing sports, biking is one of the most popular ones. Therefore, creating a software specialised in this common activity has become an appropriate idea to go forward people needs.
The purpose of Best Bike Paths project is to create a software system which can help cyclists record, analyze and share information about different bike paths. The application aims to build an up-to-date and collaborative inventory of cycling routes, while allowing users to keep track of their cycling performance over time.
Furthermore, the system gives the opportunity to share information about path conditions, such as the presence of potholes, and also compare the different tracks by visualizing them on a map. Best Bike Paths collects data in order to rank the bike paths comparing their final score. Indeed, the application gives registered users the opportunity of publishing data, whilst the non-registered ones can just exploit the path rankings.
Moreover, Best Bike Paths integrates external meteorological data, in order to make it disposable for all cyclists who want to go on that specific track. All of this, to give users a comfortable experience when cycling.

1.1.1 Goals

G1: Allow registered users to keep track of their personal performance and progress by recording information of their trips.

G2: Let registered users record and gather information about cycling path conditions and meteorological ones.

G3: Enable all users (both registered and non-registered) to explore and compare alternative bike paths between a starting point and a destination on a map, ranked by their quality score.

1.2 Scope

The Best Bike Paths system allows registered users to record their cycling activities, view performance statistics (such as total distance, average speed, and other metrics) and access detailed information about bike routes.
Moreover, registered users can learn more information about their progress and obtain statistics such as total distance covered, average speed, and other performance metrics. In this way, they can improve their physical skills and upgrade their performance.
Each route includes data collected both automatically, through GPS and sensors (accelerometer and gyroscope), and manually, through registered users' input (ex. path conditions). The system also provides the integration of meteorological information retrieved from external services, in order to guarantee a more complete view of each trip. Furthermore, Best Bike Paths has to check whether the paths are ideal for cycling, so if the tracks are reserved for bikes, if there is too much traffic and if speed limits are compatible with cyclists' needs.
Best Bike Paths also enables everyone (both registered users and non-registered ones) to view and compare different paths between a starting point and a destination. They are ranked according to a score, which is computed according to path conditions and efficiency. The information collected by all the users is then validated and merged to ensure data accuracy, giving more weight to most recent and recurrent data.

1.2.1 World Phenomena

WP1: People practising cycling activities on proper paths.

WP2: There are cycling paths or roads where cars are not allowed or where they must respect specific speed limits.

WP3: Mobile devices collect data through sensors like GPS, accelerometer and gyroscope.

WP4: External services provide maps and information about meteorological conditions.

WP5: Cyclists observe and pay attention to the condition of the bike path they are riding on.

WP6: Over time, the condition of bike paths may change due to environmental factors, maintenance or traffic.

WP7: Weather and environmental conditions influence cyclists' trips and path safety.

1.2.2 Shared Phenomena

World controlled

SP1: Registered users record their trip to evaluate their performance.

SP2: Registered users validate or dismiss the information provided automatically by the application (through sensors like GPS, gyroscope and accelerometer) about path conditions.

SP3: Registered users manually insert information about path conditions in the system after having completed the route.

SP4: Registered users can make the information about path conditions public or private, selecting a dedicated option on the system.

SP5: Both registered and non-registered users can specify an origin and a destination to request the system to display available bike paths between the two points.

SP6: Both registered and non-registered users view, on a map, the different paths proposed by the system.

SP7: Both registered and non-registered users can see problems and scores of each path by consulting the system, selecting a path of their interest.

Machine controlled

SP8: The system shows different available paths to both registered and non-registered users, ranking them on their scores.

SP9: The system acquires information from registered users' mobile devices (GPS, accelerometer, gyroscope).

SP10: The system merges different users published problems (basing on their severity and on their timestamp) to advise other users on their path's current conditions.

SP11: The system makes the users' performance statistics available.

1.2.3 Machine Phenomena

MP1: The machine computes the score for each path, based on its status and on how effective the path is in reaching the destination.

MP2: The system orders the different available paths (between a starting point and a destination) ranking them by their score.

MP3: The system computes statistics on users' performance over time.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Registered user**: a person who has successfully created an account on the Best Bike Paths platform.
- **Non-registered user**: a person who hasn't completed the procedure of registration on Best Bike Paths platform. He can visualize suggested paths from one point to another and also published path's information.
- **Cycling paths/tracks**: places used exclusively for cycling, cycle paths or particular streets where cars are rare or where speed limits are compatible with bike ones.
- **Trip**: single cycling activity recorded by a registered user, containing data such as distance, duration, speed, and path conditions.
- **Path condition**: current state of a certain bike path. It represents its overall state, the eventual presence of potholes and its maintenance needs.
- **Score**: numeric value which is assigned by the system to each path. It represents the quality and the efficiency of the path, and it is based on user-provided data.
- **Sensor-data/automatically detected data**: data automatically acquired by the system, such as average speed, position or presence of path discrepancies by using different sensors, such as GPS, accelerometer and gyroscope. This feature is useful not only for tracking path conditions, but also for evaluating users' performance.
- **External service**: third party systems used by Best Bike Paths, which give information about meteorological conditions and provide updated maps.
- **Meteorological information**: this is environmental data, useful when going on a specific path.

1.3.2 Acronyms

- **BBP**: Best Bike Paths

1.3.3 Abbreviations

- **G**: goal
- **WP**: World Phenomena
- **SP**: Shared Phenomena
- **MP**: Machine Phenomena
- **stm**: state machine

1.4 Revision History

- Version 1.0: December 23, 2025

1.5 Document Structure

1. **Introduction**: a concise overview of the project derived from the analysis of the provided specifications. This section outlines the goals of the product, its scope and its phenomena.
2. **Overall Description**: a more detailed presentation of the product that also comprehends real world scenarios and examines the system from a technical and functional perspective.
3. **Specific Requirements**: a detailed enumeration of all requirements needed in order to achieve the goals, and their relationship with domain assumptions.
4. **Formal analysis using Alloy**: a formal representation of a selected subset of real-world phenomena modeled through Alloy language. This is aimed at identifying potential inconsistencies between assumptions and the previously defined phenomena.
5. **Effort spent**: a detailed account of the time spent by each team member involved in the realization of this document.
6. **References**: a list of articles and tools consulted during the preparation of this document.

2 OVERALL DESCRIPTION

2.1 Product perspective

2.1.1 Scenarios

User-driven scenarios

Registered user activates automated mode to log trip performance and detect path anomalies

A registered user RU, before starting his cycling activity, activates the recording function on the BBP application on his mobile device. While RU keeps cycling, BBP keeps collecting data (such as distance covered, duration, average and maximum speed, presence of potholes) thanks to the sensors present on RU's mobile device. Once RU has completed the trip, he stops the BBP's recording activity on his mobile device.

Registered user manually inserts path information

A registered user RU decides to publish information about the traveled path P and wants to use the manual procedure of insertion. So, after clicking the insertion button, RU starts compiling the form, which requires the origin and final destination of P, the type of road (paved roads, dirt tracks, and dedicated bike lanes), the presence of irregularities on P and its location (street names or cross roads). Then RU checks all the information and saves it.

Registered user eventually corrects and confirms automatically detected data

After the registered user RU has completed the trip T, BBP system shows a list of all the automatically generated data about path status (such as presence of potholes, surface status, type of road). RU can choose to confirm them without changing them, to dismiss them, or to correct them. In this scenario, RU modifies the false positives about potholes, and then confirms the data.

Registered user dismisses automatically detected data

After the registered user RU has completed the trip T, BBP system shows a list of all the automatically generated data about path status (such as presence of potholes, surface status, type of road). RU can choose to confirm them without changing them, to dismiss them, or to correct them. In this scenario, RU dismisses the data, which will be deleted.

Registered user publishes his path information

After confirming the automatically detected or manually inserted information, the registered user RU decides to publish it in order to make it disposable to other users. So, RU goes to the saved path information and then clicks the publish button to share it on the BBP platform.

Registered or non-registered user selects a starting point and a destination to see the possible paths

Before going on a trip, a user U (both registered or non-registered) decides to consult the application to see if there are different options to reach the destination D. U goes into the dedicated section to look for available paths from origin O to D. After inserting the desired O and D, U visualizes on the map different tracks ordered by a precise score. This score will be useful for U, to decide the most efficient path.

System-driven scenarios

The system integrates a registered user's trip with meteorological data

When a registered user RU records a trip, the BBP system automatically retrieves meteorological data about that location (such as temperature, wind speed, weather conditions) from an external service and attaches them to the trip data. This allows RU to access complete information about the trip environment.

The system merges and validates the registered users information

The BBP system receives new publishable information about a path P from a registered user RU. The BBP system immediately checks if there is some older information (also from other registered users) about P. If this condition is satisfied, BBP merges the data by considering the freshness of all data points and the number of confirming entries.

2.1.2 Domain class diagram

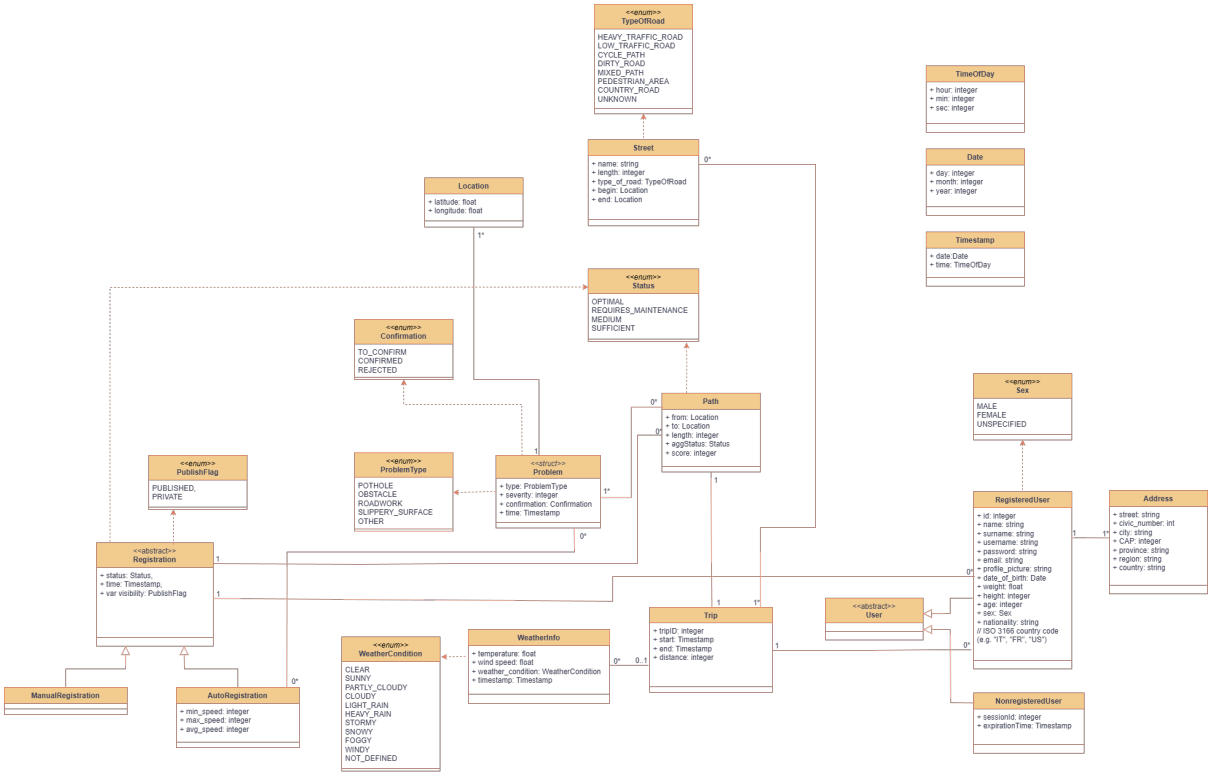


Figure 1: Domain Class Diagram of the BBP system

This Domain Class Diagram represents all the entities involved in the domain of our system and the relationships between them.
Here there are some observations about our class-diagram:

- The entities `TimeOfDay`, `Date` and `Timestamp` are useful for specifying temporal attributes within other entities, whenever there is a need to represent time-related information.
- There is an abstract class `User`, which is extended by `RegisteredUser` and `NonregisteredUser`. The `RegisteredUser` includes a richer set of attributes, as it can fully access all the features of the application. Therefore, it is important to register this type of user and store the relative data.
- The entity `NonRegisteredUser` is represented as well. Although it is not permanently stored in the database, it causes temporary data to be saved in memory in a non-persistent way, such as the session ID (`sessionId`) and the timeout (`expirationTime`).
- The attribute `profile_picture` is described as a String, because it is assumed that the image is an URL.
- The entity `Problem` is related to several core entities of the system. It is associated with `Trip`, since a trip maintains a list of problems that have been detected and then confirmed by the user. It is also connected to `AutoRegistration`, which stores a list of automatically detected problems that require user confirmation. Finally, `Problem` is linked to `Path`, as it represents the set of problems that are currently active on a given path at a specific moment, meaning that they have not been fixed by another trip reporting an optimal path condition.

2.1.3 State diagrams

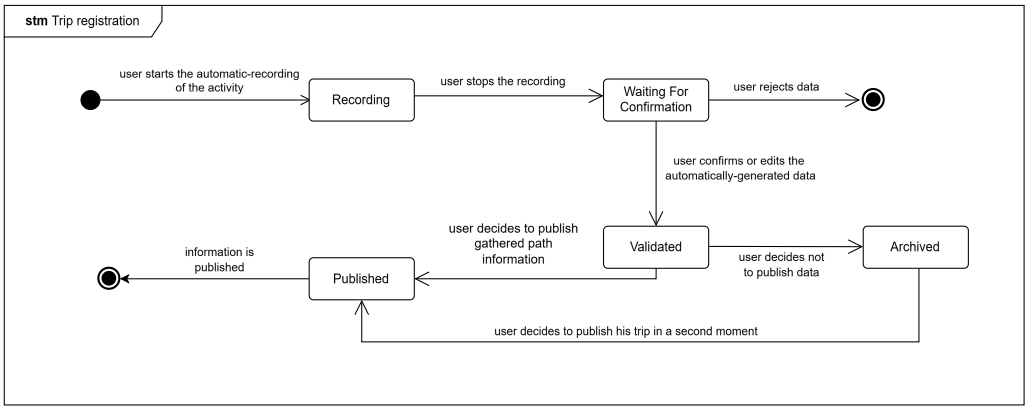


Figure 2: Trip registration

Everything starts when a user activates the automatic recording to track their journey. While the user is moving, the system remains in the *Recording* state. Once the user stops the activity, the system moves to a transitional phase called *Waiting For Confirmation*. During this time, the user has total control: they can either reject the data entirely (which ends the process) or they can review and edit the details to reach the *Validated* state. From the validation point, the user faces a choice. They can decide to share their journey immediately, moving it to the *Published* state, or they can choose to keep it private for the time being. If they choose the latter, the trip is *Archived*. An archived trip, anyways, can be published later by the user. The process is complete once the trip information is officially published for others to see.

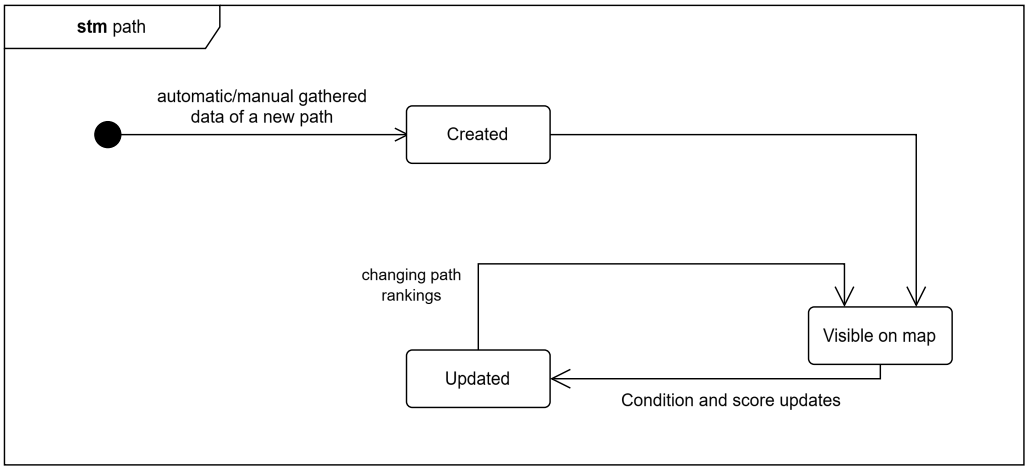


Figure 3: Path creation and score update

A path's life begins when data is gathered, either manually by a contributor or automatically by the system, resulting in a *Created* status. After the creation, it moves to the most important state: *Visible on map*, in this way, other users can see it during path search. The path remains active and visible, but it is also subject to constant evolution. Whenever new information regarding the path's condition or user scores is received, the system moves the path into an *Updated* state. In this phase, the system recalculates rankings and refreshes the path's profile. Once these changes are processed, the path returns to be *Visible on map*, ensuring that users always have access to the most accurate and up-to-date cycling information.

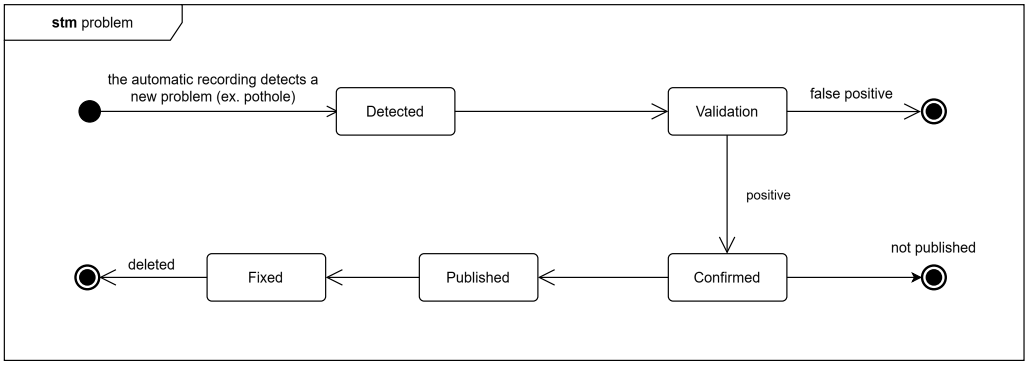


Figure 4: Problem detection and resolution

The process begins when the system's sensors during an automatic recording identify a potential issue, such as a pothole. At this initial stage, the problem is marked as *Detected*. It isn't shared immediately; instead, it enters a *Validation* phase where the data needs to be manually checked by the user. If the user doesn't confirm the problem, it is flagged as a "false positive" and the process ends. Otherwise, it moves to the *Confirmed* state. If the user decides to publish the problem, it is just added to the Active Problems of that Path and it passes to the state *Published*. If it is not published, the flow stops. Once the physical problem has been repaired, the status shifts to *Fixed*, and eventually, the record is *Deleted* from the Active Problems list to keep the database clean.

2.2 Product functions

2.2.1 Create account and Login

This function enables users to create an account (becoming a `RegisteredUser`) on the BBP platform and/or authenticate to gain full access to the complete set of offered functionalities, such as trip registration, manual data insertion and the ability to publish their own collected path information to the community.

2.2.2 Manual insertion of Bike Paths Data

This feature contributes to building and updating the collaborative inventory of cycling routes. The system allows registered users to manually input information regarding a cycling path. This includes specifying the street names and the observed current status of the path (optimal, requires maintenance, medium, sufficient).

2.2.3 Automatic Data Acquisition

This is the core function for tracking cycling activities. It enables the system to automatically acquire and process data from a registered user's mobile device during a trip:

Gps Tracking
The system collects location data to reconstruct the followed path and calculate performance metrics such as total distance and speed. Moreover, it enables accurate identification of the relevant area to retrieve meteorological information.

Sensor Data Acquisition
The system acquires data from the mobile device's accelerometer and gyroscope to keep track of significant movements that may suggest the presence of potholes or other obstacles on the path.

Meteorological Data Acquisition
The system acquires meteorological data through a dedicated external service. The average geographical location of the user is determined using GPS Tracking. Based on this location, the system retrieves the main environmental parameters, by using an external service, including: temperature, wind speed, humidity, weather conditions (e.g., sunny, cloudy, rainy).

2.2.4 Bike Paths visualization on Map

This function is accessible to both registered and non-registered users. It allows the user to specify an origin and a destination and then to visualize on a map all available bike paths between these two points. If multiple paths exist they are overlayed based on their score.

2.2.5 Score Calculus

This is a critical system function responsible for ensuring path quality. For each path, the system computes a score that reflects both the overall condition of the path and its length. The computation starts by assigning a quality value derived from the aggregated path status, which summarizes the condition of the path based on available information. Then, a penalty related to the path length is applied, so that longer paths progressively reduce the overall score. The resulting value is finally normalized within a fixed range, ensuring that the score cannot fall below a minimum value nor exceed a maximum one. This approach allows the system to fairly compare different paths and to rank them according to both their quality and their efficiency.

2.2.6 Paths information data filtering, cleaning, merging

This function ensures the reliability and accuracy of the public path inventory. The system must:

- Facilitate the correction or confirmation by the registered user of any automatically detected obstacles (e.g., dismissing false positives).
- Consolidate and merge path information contributed by multiple users. This process must prioritize data based on its freshness (timestamp) and the number of confirming entries received from the community.
- Validate whether a street/path is suitable for cycling (e.g, low traffic, compatible speed limits).

2.3 User characteristics

BBP system is designed to support two primary classes of users, each one having a distinct set of privileges.

2.3.1 Registered User

The Registered User is a person who has successfully created an account on the BBP platform. This class of user is the main contributor and beneficiary of the system's features. He is an active contributor. His activities consist of:

- **Account Management and Performance Tracking**: The user must be able to create an account and authenticate to gain access. Once logged in, he can register and store his individual trips (`Trip` entity), allowing him to visualize his personal performance statistics over time, such as total distance covered and average speed.

- **Path Information Contribution**: He is responsible for feeding the system's collaborative inventory. This involves manually inserting data about paths (street names and status) or using the automated mode to acquire sensor data.
- **Data Validation and Publishing**: Following automated data acquisition, the user must correct or confirm the automatically detected data (e.g., dismissing false positives about potholes). Once information is validated or manually inserted, the user has the possibility to publish it, making the path information available to the entire community.
- **Path Consultation**: Like non-registered users, he can consult and compare available bike paths on a map between a chosen origin and destination. Moreover, he can see the ranking of the available paths, from the best-scored to the worst-scored.

2.3.2 Non-Registered User

The Non-Registered User is a person who accesses the BBP platform without having completed the registration procedure. This user class benefits from the collaborative data gathered by the registered community. They are able to use the search and visualization capabilities of the platform to plan their trips. This involves specifying an origin and a destination and requesting the system to display available bike paths between those two points on a map. They can then consult the path rankings and scores derived from the community's published data to decide the most efficient or highest-quality route.

2.4 Assumptions, dependencies and constraints

2.4.1 Regulatory policies

- **Data Minimisation and Purpose Limitation**: All personal information collected (including user ID, email, full name and performance statistics) must be strictly necessary for providing BBP services.
- **Location Data Handling**: GPS data, trip recordings and detailed paths constitute sensitive personal data and must be handled with the highest level of privacy, including encryption and strict access control.
- **User's Consent**: Registered users must provide a formal consent for the collection and processing of their data, particularly before their path information is made public.
- **Non-Commercial Use**: The personal data, trip data and derived performance statistics must not be used for commercial purposes (e.g., selling data to third parties) unless explicitly consented to by the user under separate terms.

2.4.2 Domain Assumptions

D1: All registered users provide valid and accurate personal information when signing in.

D2: Manually entered path data is assumed to be accurate, including all street names and geographical references, which are expected to exist and be correctly spelled by the users.

D3: Registered users' devices are featured with functional GPS, accelerometer and gyroscope sensors.

D4: Mobile's device GPS and sensors (accelerometer and gyroscope) are working properly and they are precise enough.

D5: The BBP system has users' permission to access the data produced by device's sensors.

D6: Map cartography is assumed to be reliable and generally up to date, the service may be temporarily unavailable.

D7: Weather information is assumed to be reliable, the service may be temporarily unavailable.

D8: Data collected automatically through sensors (accelerometer, gyroscope) may include noise or false positive, it is assumed that users will carefully check and validate these data before publication.

D9: The speed threshold used by BBP is assumed to be sufficiently large to cover all the possible speeds reachable with cycling activities.

D10: When merging publishable information from multiple users, it is assumed that timestamps are reliable and comparable to determine the freshness of the data.

D11: Users searching for paths between an origin and a destination are assumed to provide valid and unambiguous geographical points.

D12: Network connectivity is assumed to be sufficient for BBP to upload trip data and retrieve map information.

3 SPECIFIC REQUIREMENTS

3.1 External Interface Requirements

3.1.1 User Interfaces

Here is shown a mockup of the application UI. The images outline the user interface of the most important screens from the point of view of both the registered user and non-registered user.

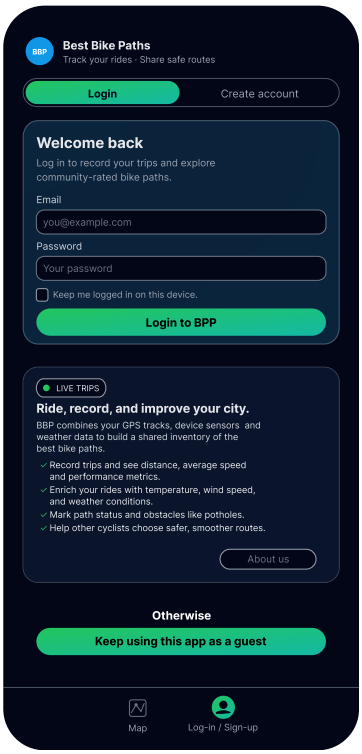


Figure 5: User login

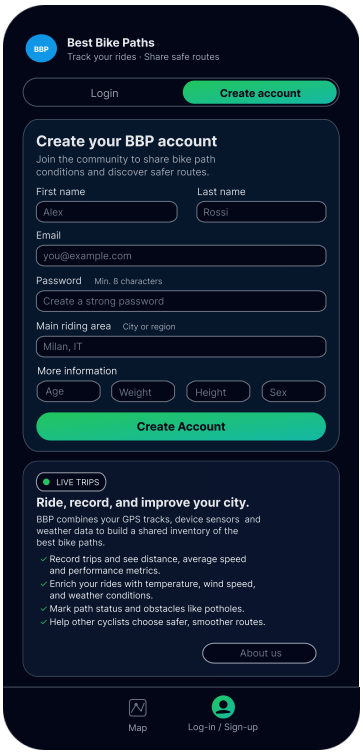


Figure 6: Account creation

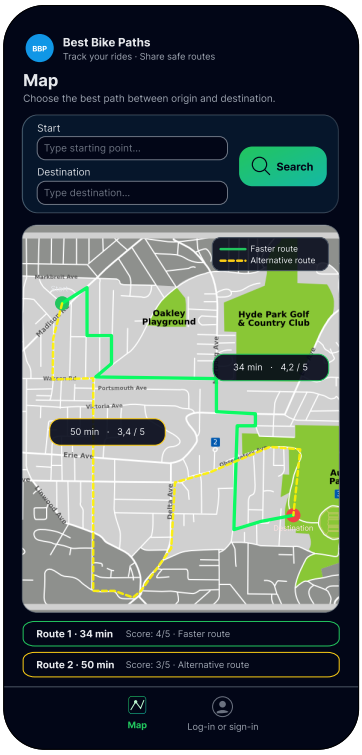


Figure 7: Map visualization (for non-registered or non-logged-in users)

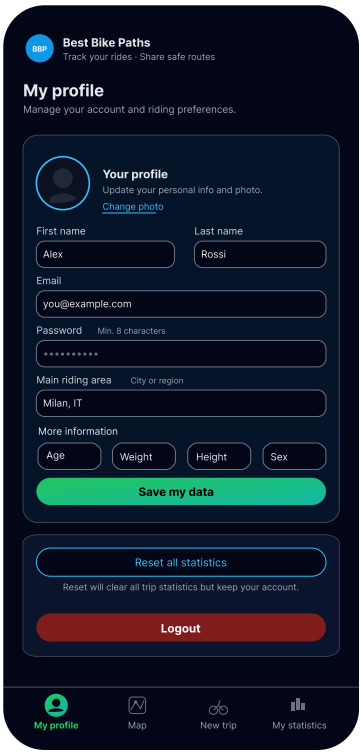


Figure 8: Profile edit and visualization

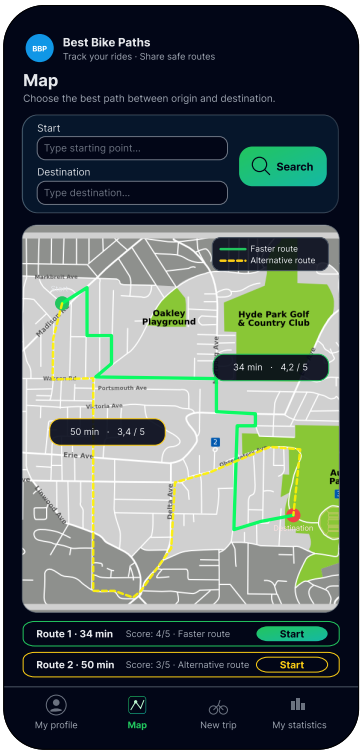


Figure 9: Map visualization (for logged-in users)

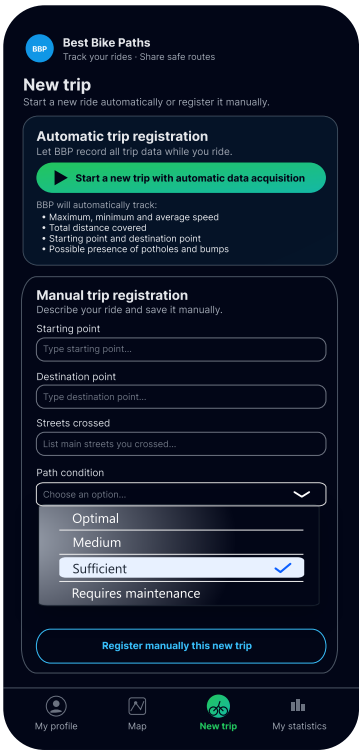


Figure 10: Start a new trip registration (in automatic mode or in manual mode)

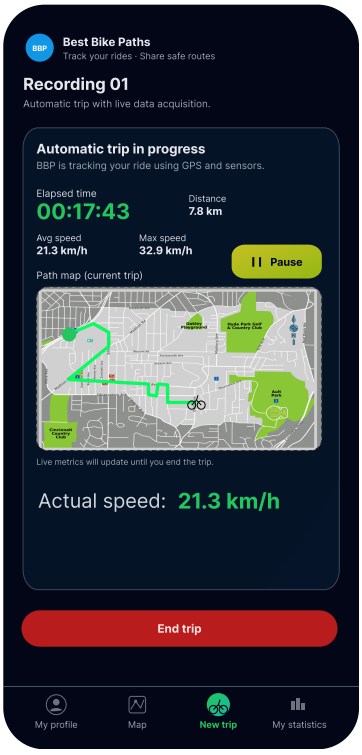


Figure 11: Recording a trip in automatic mode

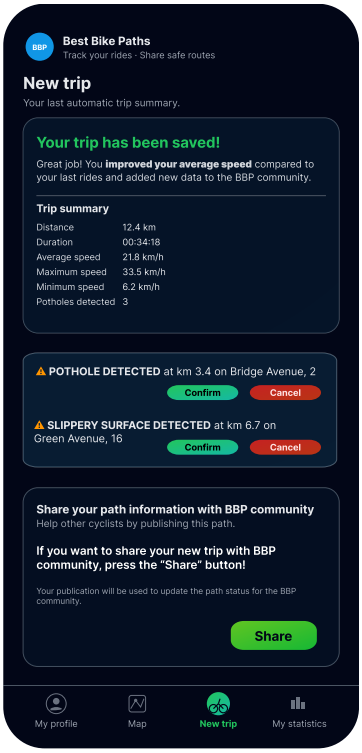


Figure 12: End of trip registration and publication



Figure 13: Visualization of personal statistics

3.1.2 Hardware Interfaces

The BBP platform is primarily designed as a mobile-friendly application, accessible through any device equipped with a modern store for third-parts applications, such as smartphones or other portable devices (like smart watches) provided with a gyroscope, an accelerometer and a GPS. Further than these requirements, an internet connection is required to ensure the full functionality of the product, even though some features still can be available in offline mode.
The platform may also interact with external services when available, such as weather information providers or map providers, using standard network interfaces, so there is no need for additional hardware.
All sensor-based data collection occurs through the device's native hardware abstraction layers, ensuring compatibility across a wide range of mobile devices.

3.1.3 Software Interfaces

External Weather Service Interface

BBP integrates with external meteorological services to enrich trip records with contextual weather data. Through a standardized API, the system retrieves information such as temperature, wind speed and current weather conditions. This interface provides users a more complete insight about their cycling activities.

Mobile Sensor Interface

To support automated data acquisition during bike trips, BBP interacts with the software layers that expose mobile device sensor data (GPS, accelerometer, gyroscope). Through the device's operating system APIs, the platform accesses GPS coordinates, accelerometer readings and gyroscope data. These interfaces enable the system to reconstruct the user's route and detect potential road issues such as potholes or sudden strange movements, which users can later confirm or correct.

Mapping and Geolocation Interface

BBP relies on third-party map services to visualize bike paths, compute routes between two points and evaluate alternative paths. These services provide map rendering, path time calculation and geospatial data, which are essential for displaying the routes, scored by path quality and effectiveness. Moreover, this service makes available information about cycling paths, for example providing speed limits or traffic information.

Authentication and User Management Interface

While the platform allows public access for path browsing, registered users must authenticate to store trips, insert path information and manage publishability settings. The system interacts with a dedicated authentication module, in order to manage all user-related access and registration procedures. This interface handles account creation, secure login, session management and the assignment of appropriate access privileges, ensuring privacy and the permission to access to registered-user functionalities.

Data Aggregation and Validation Interface

To combine publishable information collected from multiple users, BBP uses internal components dedicated to merging data and consistency checking. This interface coordinates inputs from different data sources (manual inserted paths, automated sensor data and manually-inserted problems) and ensures that path status is determined by weighting factors such as freshness and the number of confirming reports.

3.1.4 Communication Interfaces

The BBP platform, as an application, communicates with users and external services through secure internet protocols. All interactions (such as trip data transmission, sensor data upload, and retrieval of map or weather information) must occur over encrypted channels using HTTPS to ensure confidentiality and integrity.
During automated trip recording, the mobile client maintains a continuous interaction with the device's GPS, collecting location data at short and regular intervals. However, the collected GPS and sensor data are stored locally on the client and transmitted to the BBP server only at the end of the trip (always with HTTPS).
Communication with external meteorological services and mapping providers occurs through standardized REST APIs, enabling the platform to perform requests and to receive detailed responses, with all necessary meteorological data.

3.2 Functional requirements

All the functional requirements are organized in subsections and each subsection is the representation of a different functional area.

User Registration, User Login and User Account Management

- **R1**: The system allows users to create an account, inserting their personal data.
- **R2**: The system allows registered users to log in securely (it validates user accounts through secure authentication mechanisms).
- **R3**: The system allows users to update and modify their personal information.
- **R4**: The system allows users to log out from their account.
- **R5**: The system ensures that only authenticated users can access functionalities reserved for registered members.

Trip Recording & Statistics

- **R6**: The system allows registered users to automatically record a bike trip by collecting GPS, accelerometer and gyroscope data.
- **R7**: The system calculates trip statistics such as total distance, duration, average speed and performance metrics.
- **R8**: The system retrieves weather information from external services to enrich trip data as soon as they are available.
- **R9**: The system stores recorded trips in the user's personal statistics, enabling the history trip view.

Path Information Insertion & Management

- **R10**: The system allows registered users to manually insert information about bike paths, including street names, status, and possible problems.
- **R11**: The system allows registered users to start an automatic trip detection process, during which it automatically gathers information such as street names, path status, covered distance, speed and potential issues.
- **R12**: The system detects potential issues during automated trip recording.

R13: The system allows users to make their path information publishable and available to the community.

Data Aggregation & Path Status Evaluation

R14: The system aggregates publishable information from multiple users regarding the same bike path.

R15: The system evaluates the score of a bike path, based on the effectiveness in reaching a destination and its status.

R16: The system determines a final path status (e.g., optimal, medium, sufficient, requires maintenance) based on different users-provided information.

R17: The system updates the path status when new relevant information becomes available.

R18: The system automatically marks a previously reported obstacle as fixed when a recent user reports the associated path conditions as optimal.

Path Search

R19: The system allows any user, registered or not, to search for bike paths by selecting an origin and a destination.

R20: The system allows any user, registered or not, to visualize multiple possible bike paths on a map.

R21: The system ranks and displays multiple routes according to their computed score.

R22: The system allows any user, registered or not, to view details about path conditions, obstacles and aggregated status.

User Notifications

R23: The system notifies users of detected obstacles, after automated recording, in order to make them confirmed or corrected.

3.2.1 Use Case Diagram

Account management

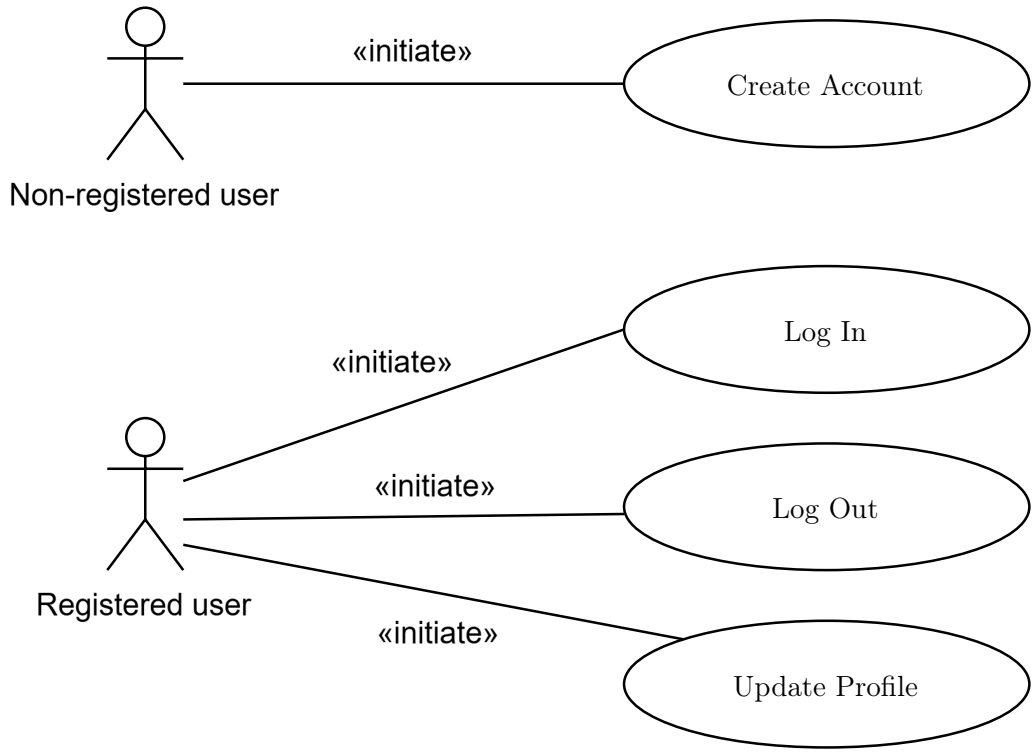


Figure 14: Use case diagram: Account management

Paths and trips management

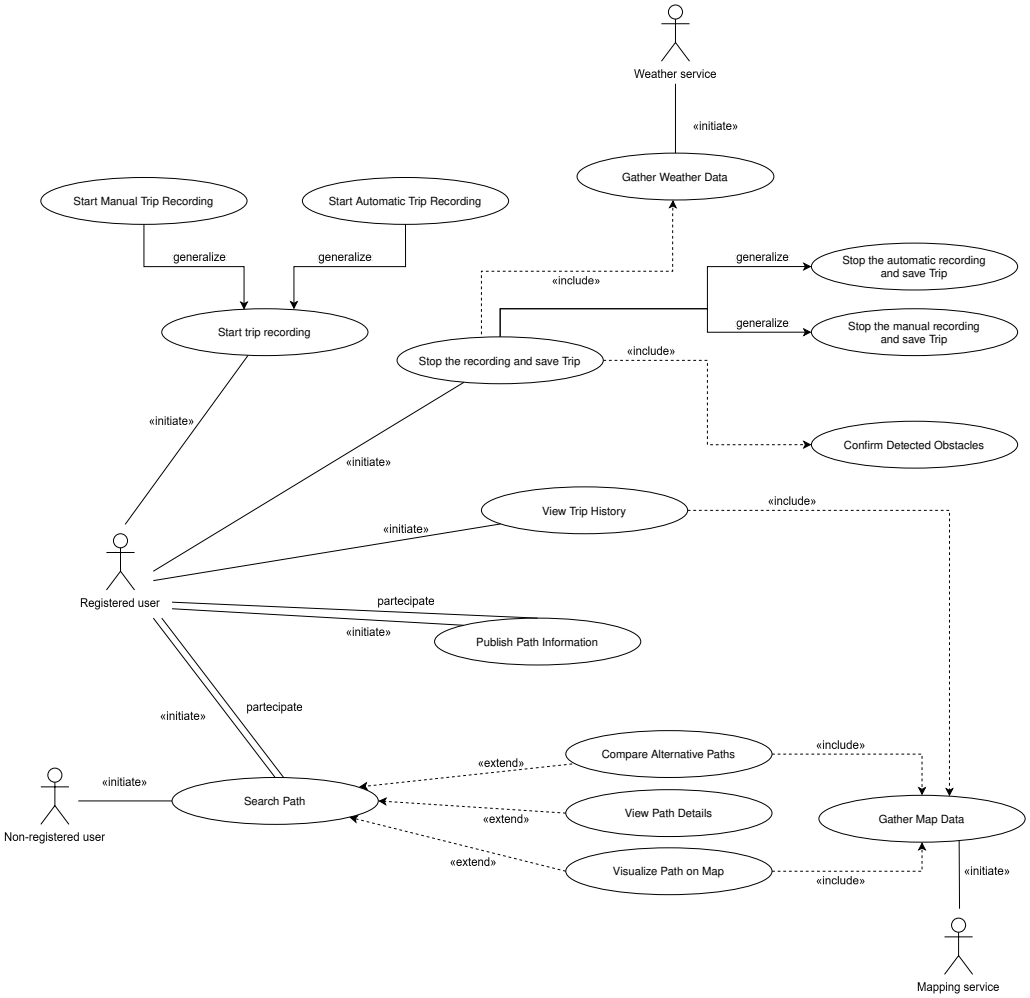


Figure 15: Use case diagram: Paths and trips management

3.2.2 Use Cases

ID	**[UC1]**
Name	Create Account
Actors	Non-registered user
Entry condition	The non-registered user opens the BBP application and selects the "Create account" option.
Event flow	(a) The user selects the "create account" function from the home page. (b) The system displays a registration form and asks for mandatory information (name, surname, email, password and main riding area). (c) The user fills in the form and submits the registration request. (d) The system validates the entered data (e.g., format of email, password constraints, uniqueness of the email, validity of the riding area). (e) If the data are valid, the system creates a new user account and stores the information.
Exit condition	The user account has been successfully created, stored in the system.
Exception	• (d) The system detects invalid or already used data (e.g., email already registered, weak password) and notifies the user, asking to correct the information before resubmitting.

Table 1: Use Case 1 - Create Account

ID	**[UC2]**
Name	User Log-in
Actors	Registered user
Entry condition	The registered user opens the BBP application and chooses the "Log-in" option.
Event flow	(a) The user enters his credentials (email and password). (b) The system receives the credentials and checks them against the stored user data. (c) If the credentials are valid, the system establishes an authenticated session for the user. (d) The system redirects the user to the "My Profile" page.
Exit condition	The user is successfully authenticated and has access to the functionalities reserved for registered users.
Exception	• (b) If the credentials are invalid, the system denies access, shows an error message and keeps the user on the login page.

Table 2: Use Case 2 - User Log-in

ID	**[UC3]**
Name	Log-out
Actors	Registered user
Entry condition	The user is currently logged in to BBP.
Event flow	(a) The user selects the "Log out" option from the "My Profile" page. (b) The system terminates the user's session and clears any session-related data. (c) The system redirects the user to the login screen.
Exit condition	The user is no longer authenticated and the access to registered-only functionalities is disabled.
Exception	-

Table 3: Use Case 3 - Log-out

ID	**[UC4]**
Name	Update Profile
Actors	Registered user
Entry condition	The user is logged in and opens the "My profile" section.
Event flow	(a) The user requests to edit their profile information. (b) The system displays the current profile data (e.g., name, contact information, preferences). (c) The user modifies one or more fields and submits the changes. (d) The system validates the updated data (e.g., required fields, email format). (e) If the data are valid, the system stores the new profile information and confirms the update to the user.
Exit condition	The user's profile information is successfully updated in the system.
Exception	(d) If the updated data are invalid (e.g., wrong format, missing mandatory fields), the system shows an error message and asks the user to correct the input.

Table 4: Use Case 4 - Update Profile

ID	**[UC5]**
Name	Start Trip Recording
Actors	Registered user
Entry condition	The user is logged in and has access to the trip recording feature.
Event flow	(a) The user opens the trip recording section and selects to start a new trip (manual or automatic mode). If the user chooses automatic mode: (b1) The system requests permission to access GPS, accelerometer and gyroscope data from the mobile device. (c1) Once permissions are granted, the system starts collecting location and sensor data at short and regular intervals, and initializes the internal trip record. (d1) The system indicates that trip recording is active (e.g., through a visual indicator or timer). If the user chooses manual mode: (b2) The user starts filling the form to insert trip information (e.g., starting point, speed, etc.).
Exit condition	A trip recording session is successfully started in automatic mode, otherwise the form is filled in the manual one.
Exception	(b1) If the user denies permissions to access the required sensors or GPS, the system notifies the user that automatic recording is not available and may offer manual trip recording as an alternative.

Table 5: Use Case 5 - Start Trip Recording

ID	**[UC6]**
Name	Stop the recording and save trip
Actors	Registered user
Entry condition	A trip recording session is currently active.
Event flow	If the user chose automatic mode: (a1) The user chooses to stop the ongoing trip recording. (b1) The system stops collecting new data and compiles all recorded information (e.g., GPS track, timestamps, sensor events). (c1) If the system has detected potential obstacles (e.g., potholes) from sensor data (which have not been confirmed yet), it presents them to the user for confirmation or correction. (d1) The user checks the detected issues and confirms, edits, or removes them. If the user chose manual mode: (a2) The user checks the filled information and then saves it. Common events: (e) The system computes trip statistics, such as covered distance, duration, average speed, and other performance metrics. (f) The system saves the trip, including track, statistics and any confirmed path issues, into the user's trip history and, if chosen by the user, marks related path information as publishable.
Exit condition	The trip is successfully stored, complete with statistics and validated obstacle information and becomes available in the user's trip history.
Exception	(b1-d1 / a2) If the user cancels the saving process, the system may either discard the trip.

Table 6: Use Case 6 - Stop the recording and save trip

ID	**[UC7]**
Name	Gather Weather Data
Actors	Weather service (external system)
Entry condition	A trip recording has been just saved into the BBP system and location and/or time data are available.
Event flow	(a) The system determines the relevant parameters for the weather request, such as geographic coordinates and time interval of the trip. (b) The system sends a request to the external weather service through a secure API. (c) The weather service receives the request, processes it and returns meteorological information (e.g., weather conditions, temperature, wind speed). (d) The system receives the response and associates the weather data with the corresponding trip.
Exit condition	Relevant weather data are successfully retrieved and attached to the trip record.
Exception	(c) If the weather service is unavailable, returns an error, or responds with incomplete data, the system logs the problem, notifies the user, and stores the trip without weather enrichment.

Table 7: Use Case 7 - Gather Weather Data

ID	**[UC8]**
Name	View Trip History
Actors	Registered user
Entry condition	The user is logged in.
Event flow	(a) The user opens the "My statistics" section. (b) The system retrieves the list of trips associated with the user, including main information such as date, distance, speed values and duration. (c) The system displays the list of trips, possibly allowing filtering or sorting (e.g., by date or distance). (d) The user selects a specific trip from the list. (e) The system shows detailed information for the selected trip, including the route, statistics, weather data (if available) and any reported obstacles.
Exit condition	The user can successfully inspect their past trips and, for each, see detailed information.
Exception	(b) If no trips are found for the user, the system informs the user that no trip history is available and may suggest starting a new trip.

Table 8: Use Case 8 - View Trip History

ID	**[UC9]**
Name	Publish Path Information
Actors	Registered user
Entry condition	The user is logged in and has stopped a trip registration procedure (automatic or manual). Or the user has accessed "My Statistics".
Event flow	If the user stopped a trip registration: (a1) The system displays the ridden path's data (e.g., status, obstacles). (b1) The system asks the user to confirm the publication when he finishes a trip, and the user ticks the option to publish it. (c1) After confirmation, the system marks the selected path information as publishable and integrates it into the shared path inventory. If the user wants to publish later an older trip registration: (a2) The user visualizes the old recordings and chooses to publish one of them. (b2) The user selects the recording he wants to share. (c2) The system displays the ridden path's data (e.g., status, obstacles). (d2) The user confirms the choice and publishes the recording.
Exit condition	The selected path information is made available as publishable data for other users of BBP.
Exception	-

Table 9: Use Case 9 - Publish Path Information

ID	**[UC10]**
Name	Search Path
Actors	Registered user, Non-registered user, Mapping service (external system)
Entry condition	The user (registered or not) opens the "Map" section.
Event flow	(a) The user specifies an origin and a destination, possibly using map selection or address input. (b) The system validates the input locations. (c) The system sends a request to the mapping service and to its internal path database to compute one or more bike paths between origin and destination. (d) The system gathers the candidate paths and computes a score for each path, considering path status, obstacles and effectiveness in reaching the destination. (e) The system displays the available paths on a map, ordered by their score and shows summary information (e.g., distance, estimated time, quality). (f) The user may select a specific path to view more detailed information.
Exit condition	The user can see one or more viable bike paths between the chosen origin and destination, with their relative scores and details.
Exception	(c-d) If no suitable path can be found (e.g., invalid locations or no bike-compatible paths), the system informs the user that no path is available and may suggest adjusting the origin or destination.

Table 10: Use Case 10 - Search Path

ID	**[UC11]**
Name	Gather and Aggregate Path Data
Actors	BBP system, Mapping service (external system)
Entry condition	Path-related information is requested (e.g., during a path search) or new publishable data about bike paths have been submitted by users.
Event flow	(a) The system identifies the path segments relevant for a given area and origin–destination pair. (b) The system retrieves existing internal information about these paths, including status reports, obstacles and historical data submitted by multiple users. (c) When needed, the system sends requests to the mapping service to obtain or update base map data (e.g., street layout, bike track availability). (d) The system aggregates user-contributed information, considering factors such as freshness of reports and number of confirming data points. (e) Based on this aggregation, the system computes or updates the overall status and score of that path. (f) The updated path data are stored in the system and made available for other users' research.
Exit condition	The system maintains an up-to-date representation of bike paths, with aggregated status and obstacle information ready to be used by path search and visualization functionalities.
Exception	(c) If the mapping service or internal data sources are temporarily unavailable, the system logs the issue and continues using the most recent available data, updating them when the services become reachable again.

Table 11: Use Case 11 - Gather and Aggregate Path Data

3.2.3 Sequence Diagrams

Login

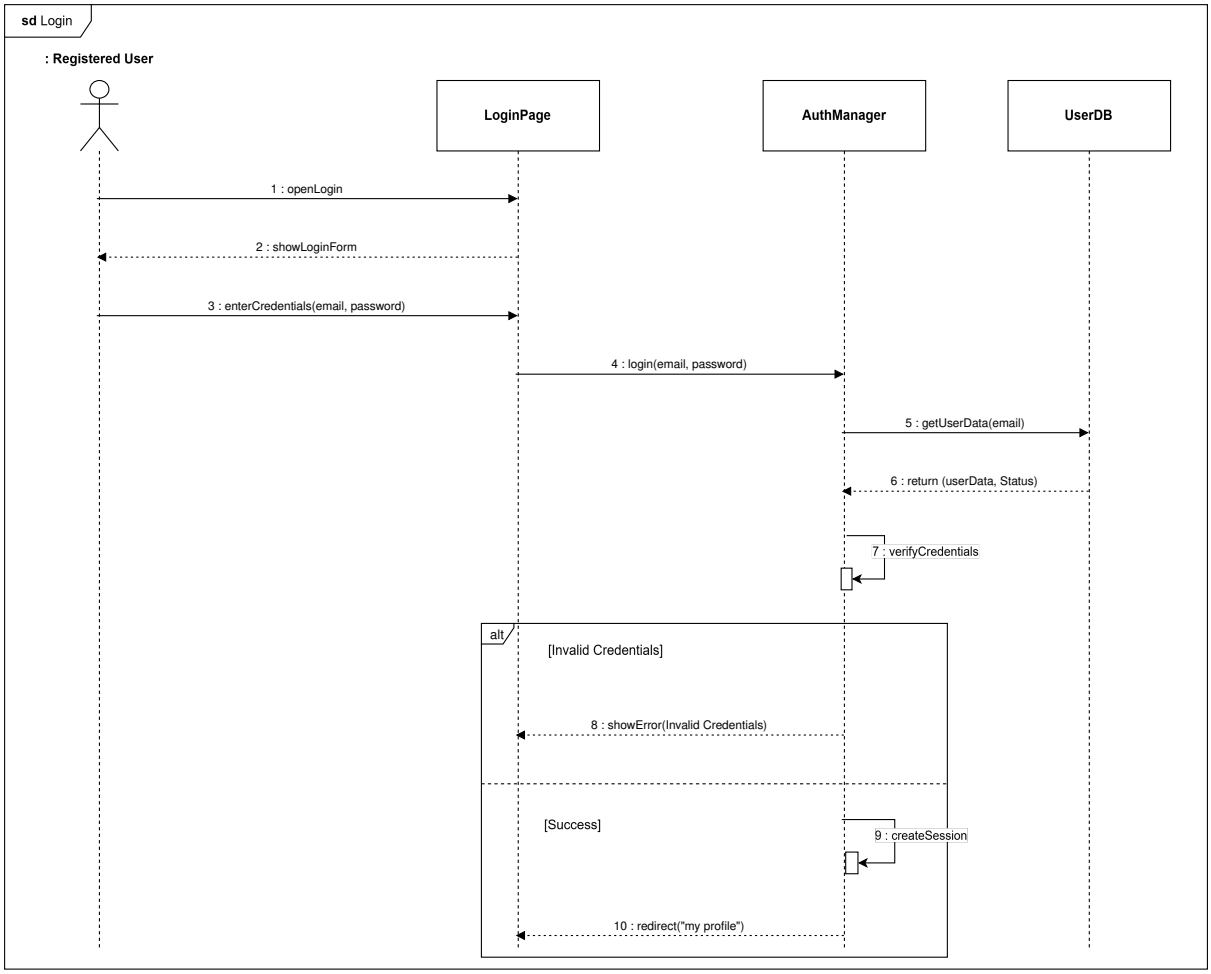


Figure 16: Sequence diagram: Login

Create account

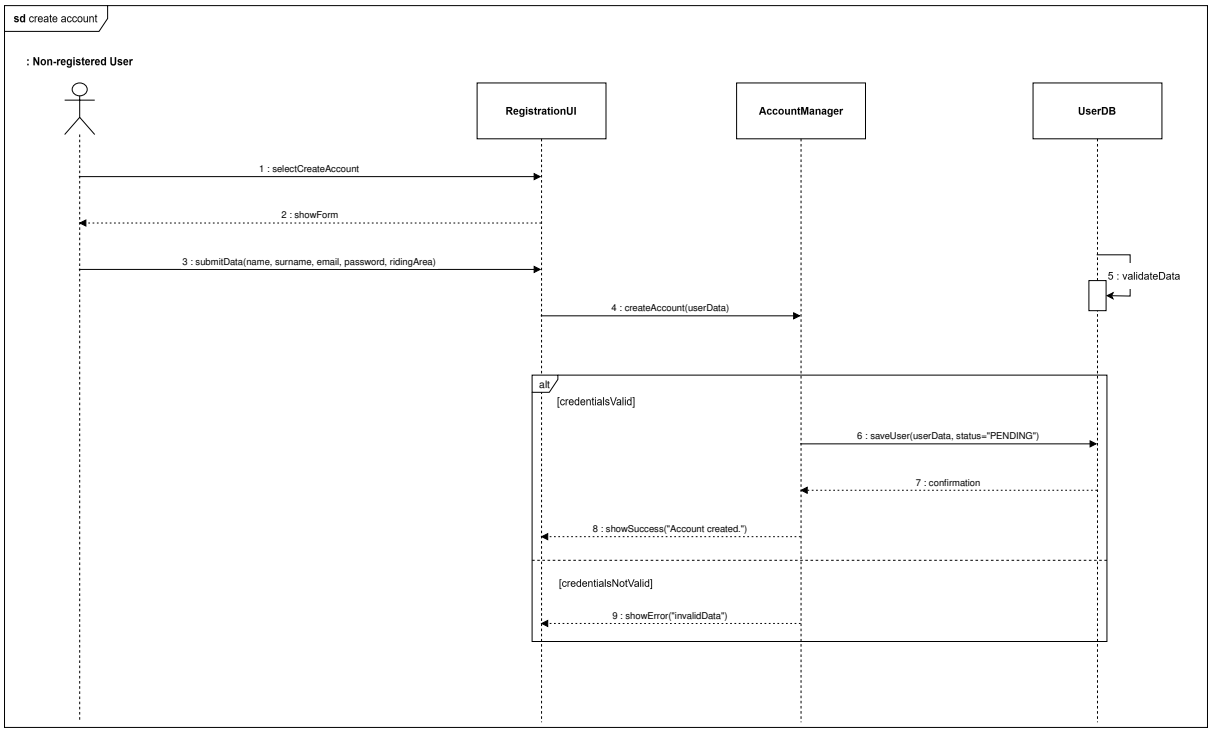


Figure 17: Sequence diagram: Create account

Automatic Trip Recording

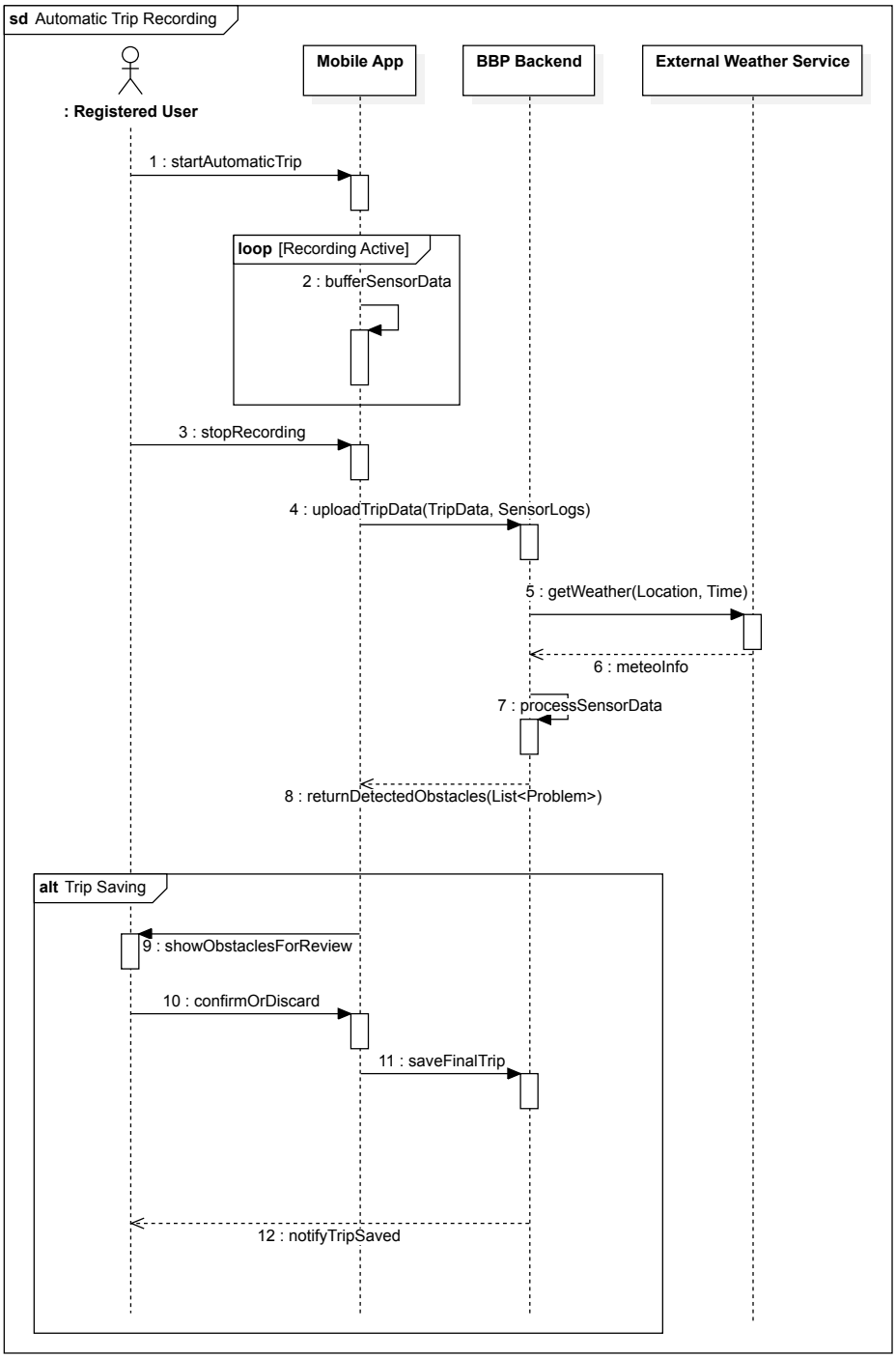


Figure 18: Sequence Diagram: Automatic Trip Recording

Manual Trip Recording

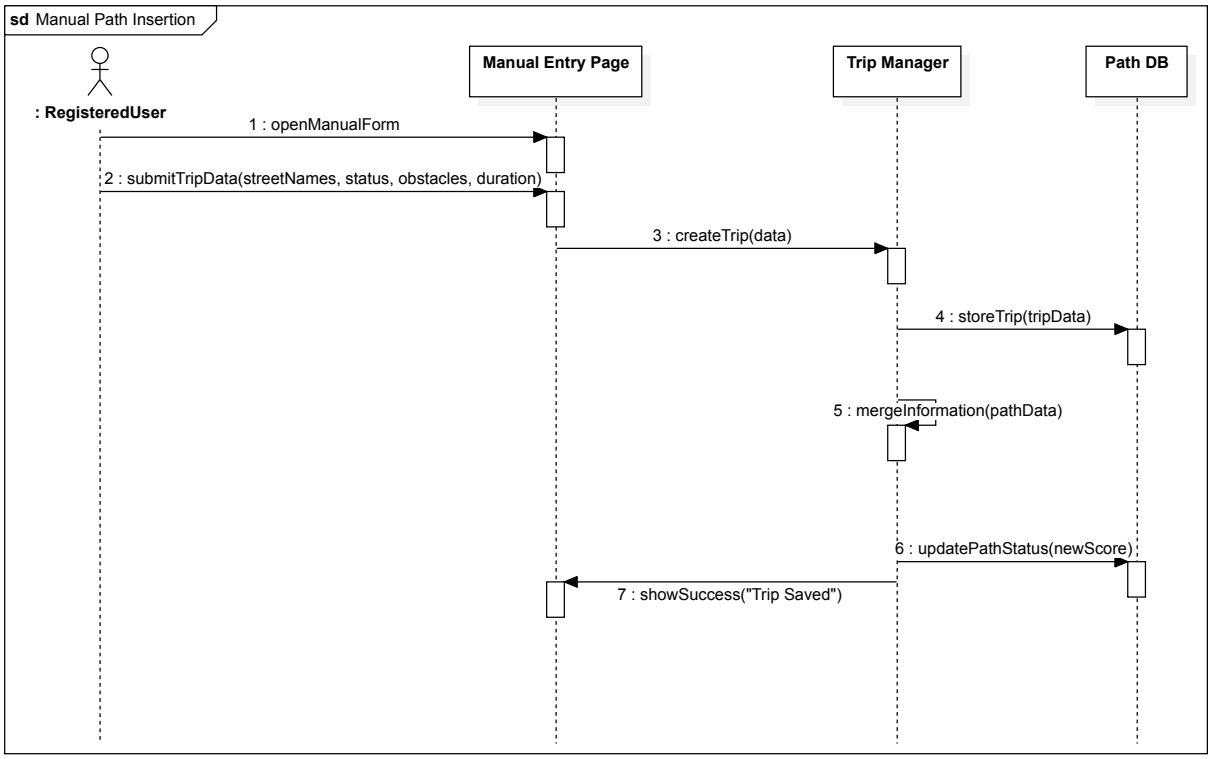


Figure 19: Sequence Diagram: Automatic Trip Recording

Path Search

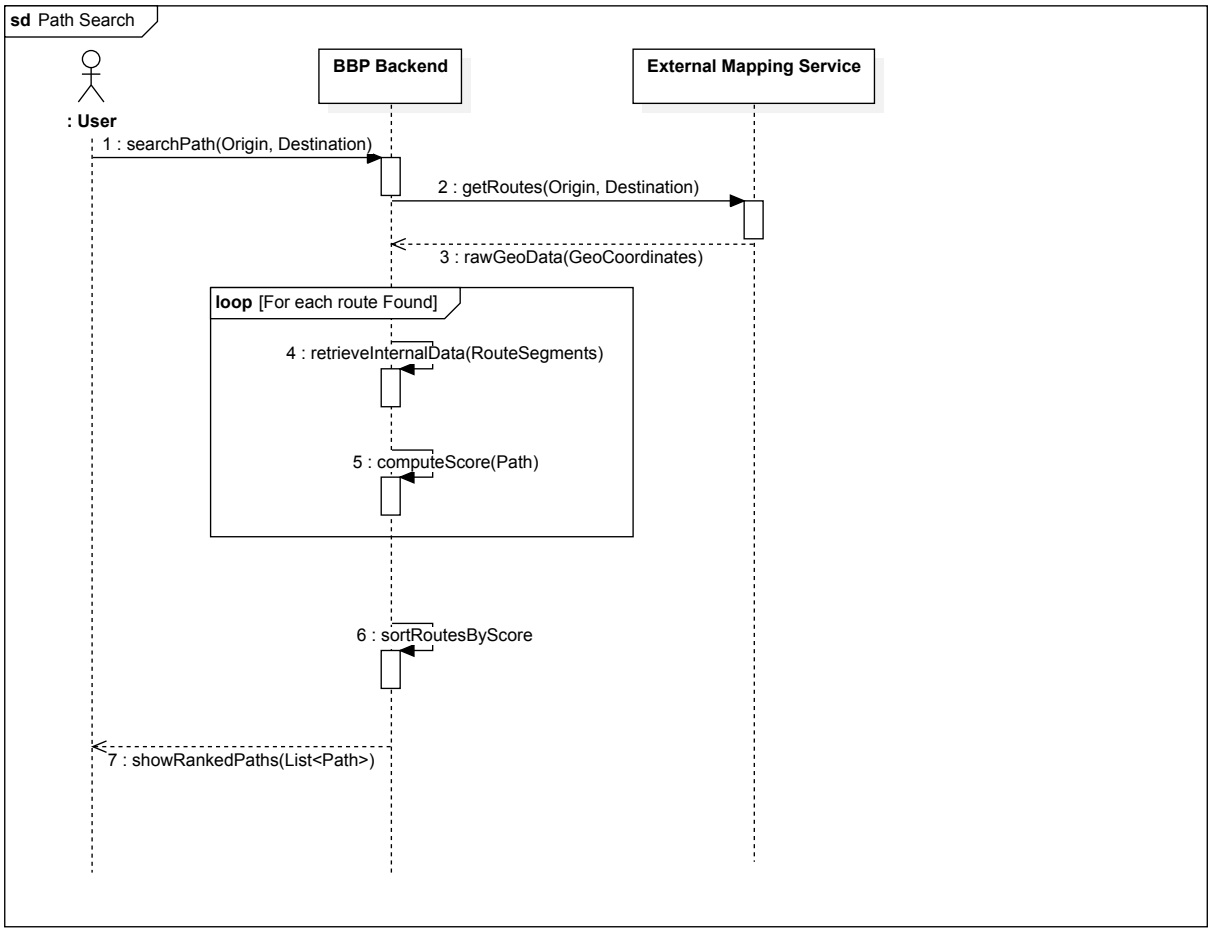


Figure 20: Sequence diagram: Path Search

Path Information Confirmation (Automatic Trip Recording)

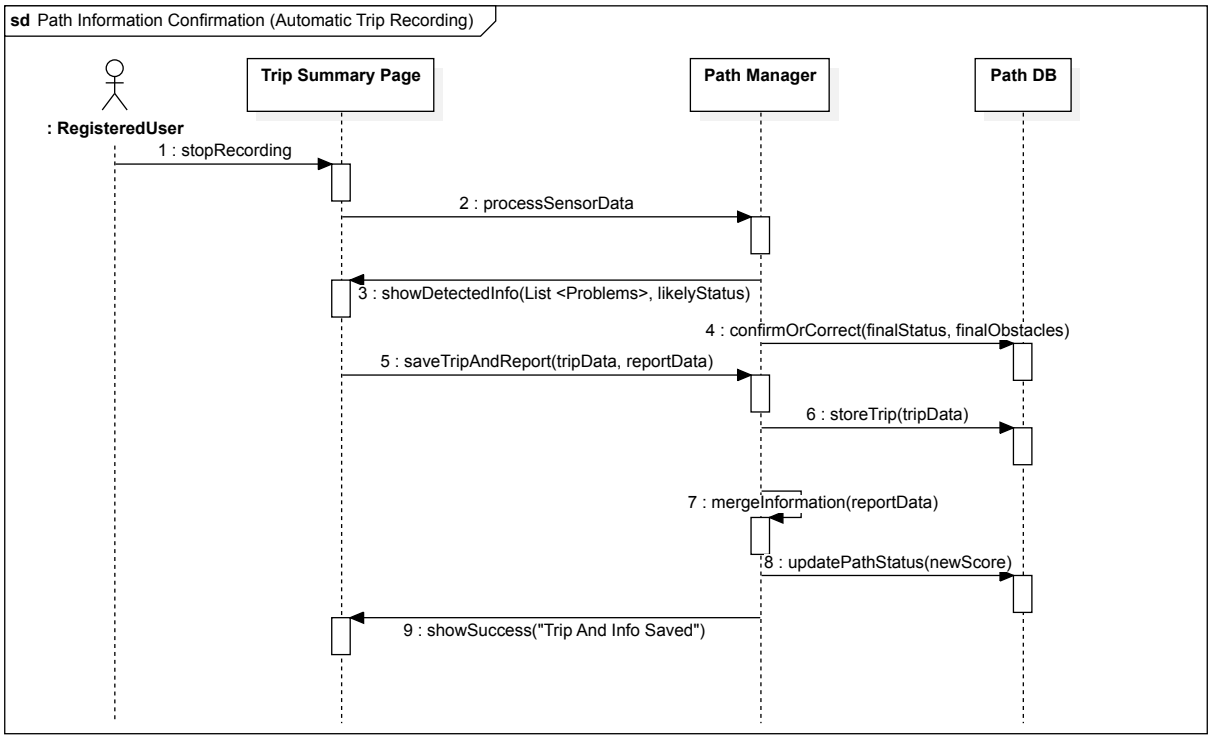


Figure 21: Sequence diagram: Path Information Confirmation

3.2.4 Requirement Mapping

The following Traceability Matrix maps each Functional Requirements (Section 3.2) to the specific Goals (Section 1.1.1) it supports.

Req.	Description	G1	G2	G3	Assumptions
R1	Create account	**X**			D1, D12
R2	User Log-in	**X**			D1, D12
R3	Update profile	**X**			D1, D12
R4	User Log-out	**X**			D12
R5	Access control	**X**			D12
R6	Automatic trip recording (GPS/Sensors)	**X**	**X**		D3, D4, D5
R7	Statistics calculation	**X**			D9
R8	Weather data retrieval	**X**			D7, D12
R9	Trip storage	**X**			D12
R10	Manual path info insertion		**X**		D2, D12
R11	Automatic path data gathering		**X**		D4, D8
R12	Issue detection		**X**		D4, D8
R13	Data publishing		**X**		D2, D12
R14	Information aggregation		**X**		D10
R15	Path Score calculation		**X**		D10
R16	Path Status determination		**X**		D10
R17	Status update on new info		**X**		D10
R18	Problem resolution (Auto-fix)		**X**		D10
R19	Path search			**X**	D11, D12
R20	Map visualization			**X**	D6, D12
R21	Ranking display			**X**	D6, D12
R22	View path details			**X**	D6, D12
R23	Notification: Detected obstacles		**X**		D12

Table 12: Requirement Traceability Matrix

3.3 Performance Requirements

Given the real-time nature of data acquisition and the dependency on external services, the system must adhere to the following performance constraints:

- **PR1 - Sensor Sampling Rate**: During the *Automatic Data Acquisition*, the mobile application must sample accelerometer and gyroscope data at a frequency sufficient to capture road irregularities (e.g., 50 Hz), managing data acquisition in a background process to ensure application responsiveness and optimizing computational load to preserve battery life.
- **PR2 - Search Latency**: The *Path Search* function must display results to the user within 3 seconds under standard 4G/5G network conditions, including the time required to query the External Mapping Service.
- **PR3 - Data Aggregation Speed**: When a user publishes a trip with new path conditions, the *Data Aggregation* process must process and merge this information into the public inventory within 5 minutes, ensuring data freshness for other users.

- **PR4 - Battery Efficiency**: The continuous use of GPS and sensors in the background must be optimized. The application should not consume more than 15% of the device's battery per hour of recording on a standard modern smartphone (approx. 4000 mAh capacity), assuming the device screen is turned off and the application is running in the background.
- **PR5 - Concurrent Users**: The BBP Backend must support at least 10.000 concurrent users performing path searches or syncing trip data without service degradation.

3.4 Design Constraints

3.4.1 Standards compliance

- **GDPR (General Data Protection Regulation)**: As the system processes sensitive location data and personal statistics, it must strictly comply with EU privacy regulations. This includes data anonymization for public path aggregation.
- **HTTPS/TLS**: All communication between the client (mobile) and the server, as well as with external services (Weather/Mapping), must be encrypted using TLS 1.2 or higher to ensure data confidentiality and integrity.
- **RESTful API**: The internal and external interfaces must adhere to standard REST architectural styles to ensure interoperability, scalability, and ease of integration with third-party services.

3.4.2 Hardware limitations

- **Sensor Dependency**: The *Automatic Trip Recording* feature explicitly requires the user's device to be equipped with a functional GPS, Accelerometer, and Gyroscope. If these sensors are missing or malfunctioning, the application must degrade gracefully, disabling automatic detection but still allowing manual trip insertion.
- **Storage Capacity**: The mobile application requires sufficient local storage to buffer trip data (Location coordinates + Sensor logs) temporarily. This is crucial to support recording in case of network unavailability during the ride without data loss.

3.4.3 Any other constraint

- **Offline-First Architecture**: The mobile application must be capable of recording a trip even in the absence of an active internet connection (e.g., in remote areas). Data synchronization with the backend must occur automatically once connectivity is restored.
- **External Service Dependency**: The system relies on third-party providers for Maps and Weather. The system must implement fallback mechanisms (e.g., displaying the trip path on a blank grid or caching previous map tiles) if the *External Mapping Service* is temporarily unavailable, ensuring the core functionality (recording) remains usable.

3.5 Software System Attributes

3.5.1 Reliability

- **Crash Recovery**: In the critical event of a mobile application crash during a trip recording, the system must be designed to persist the current state locally. Upon restarting, the application must automatically restore the recording session and save the data collected up to the point of failure, ensuring no user effort is lost.

- **Data Accuracy**: To ensure the reliability of the crowd-sourced inventory, the system must implement filtering algorithms. These algorithms must distinguish between actual road anomalies (e.g., potholes) and false positives caused by user handling (e.g., shaking the phone), filtering out irrelevant noise before data transmission.

3.5.2 Availability

- **Uptime**: The BBP system backend should guarantee a service uptime of 99.5%, with a particular focus on peak cycling hours (typically 06:00 - 22:00). High availability during these times is essential to support real-time features like automatic recording and path searching.
- **Maintenance**: Scheduled maintenance operations must be performed during off-peak hours (e.g., night time) to minimize disruption to the user base.

3.5.3 Security

- **Authentication**: Access to registered user features (e.g., saving trips, publishing data) must be protected via secure authentication tokens (e.g., JWT, OAuth 2.0) managed by the *Authentication Interface*.
- **Location Privacy**: Protecting user habits is paramount. When publishing aggregated path data, the system must apply "privacy zones" around sensitive locations inferred from user patterns (typically the start and end points of daily commutes, such as home or workplace) to prevent identification or stalking.
- **Input Validation**: All user-generated content (manually-inserted path status, profile data, path descriptions) must be sanitized on the server side to prevent common vulnerabilities such as SQL Injection or Cross-Site Scripting (XSS).

3.5.4 Maintainability

- **Modularity**: The system architecture must strictly separate the *Sensor Data Acquisition* module (mobile client logic) from the *Data Aggregation* logic (backend). This decoupling allows developers to update, for instance, the pothole detection algorithm without requiring a full redeployment of the backend infrastructure.
- **Documentation**: To facilitate future development and onboarding, all code and APIs must be documented using standard tools (e.g., Swagger/OpenAPI for REST endpoints). Code comments should adhere to standard formats (e.g., Doxygen) to enable automatic documentation generation.

3.5.5 Portability

- **Mobile Compatibility**: The mobile application component must be fully compatible with the latest major versions of mobile operating systems, specifically Android 12+ and iOS 15+, ensuring access to the latest sensor APIs.
- **Device Compatibility**: The application used for path visualization and account management must be fully functional on all modern portable devices with their operative systems, without requiring third-party plugins or extensions.

4 FORMAL ANALYSIS USING ALLOY

```
// ================================
// 4. Formal Analysis Using Alloy
// ================================

open util/integer
open util/ordering[Timestamp] as TSO

// This type is used to express complex kind of String
sig Text {}


////---------- MAIN INSTANCES ------------

sig Date {
  day: one Int,
  month: one Int,
  year: one Int
}

sig TimeOfDay {
  hour: one Int,
  minute: one Int,
  second: one Int
}

sig Timestamp {
  date: one Date,
  tod:  one TimeOfDay
}

abstract sig User {}

enum Sex {
  Female,
  Male,
  Unspecified
}

sig Address {
  street: one Text,
  civicNumber: one Int,
  city: one Text,
  cap: one Int,
  province: one Text,
  region: one Text,
  country: one Text
}

sig RegisteredUser extends User {
  id: one Int,
  name: one Text,
  surname: one Text,
  username: one Text,
  password: one Text,
  email: one Text,
  address: lone Address,
```

```
  profilePicture: one Text,
  dateOfBirth: one Date,
  weight: one Int,
  height: one Int,
  age: one Int,
  sex: one Sex,
  nationality: one Text
}

sig NonregisteredUser extends User {
  sessionId: one Int,
  expirationTime: one Timestamp
}

sig Location {
  latitude: one Int,
  longitude: one Int
}

//This enum expresses the type of road cycled
enum TypeOfRoad {
  Heavy_traffic_road,
  Low_traffic_road,
  Cycle_path,
  Dirty_road,
  Mixed_path,
  Pedestrian_area,
  Country_road,
  Unknown
}

//Status symbolises the different conditions a path can have
enum Status {
  Optimal,
  Medium,
  Sufficient,
  RequiresMaintenance
}

//It's assumed that the length is in kilometers
sig Street {
  length: one Int,
  typeOfRoad: one TypeOfRoad,
  name: one Text,
  begin: one Location,
  end: one Location,
}

// The path is an instance describing a followed track formed of
    streets
sig Path {
  streets: some Street,
  from: one Location,
  to: one Location,
  length: one Int,
  var activeProblems: set Problem,
  var aggStatus: lone Status,
  var score: one Int
```

```
}

//PublishFlag is used when publishing a rating
enum PublishFlag {
  Published,
  Private
}

//To determine whether a problem has been encountered or not
enum Confirmation {
  ToConfirm,
  Confirmed,
  Rejected
}

enum ProblemType {
  Pothole,
  Obstacle,
  Roadwork,
  Slippery_surface,
  OtherProblem
}

//Severity indicates the level of the problem
sig Problem {
  type: one ProblemType,
  where: one Location,
  var confirmation: one Confirmation,
  severity: one Int,
  time: one Timestamp
}

// Registration refers to info to publish about a path; Trip focuses on
    performance statistics
abstract sig Registration {
  by: one RegisteredUser,
  path: one Path,
  status: one Status,
  time: one Timestamp,
  var visibility: one PublishFlag
}

// Manual registration: NO problems
sig ManualRegistration extends Registration { }

// Auto registration: problems here
sig AutoRegistration extends Registration {
  var problems: set Problem,
  avgSpeed: one Int,
  maxSpeed: one Int,
  minSpeed: one Int
}

sig Trip {
  tripID: one Int,
  owner: one RegisteredUser,
  path: one Path,
  distance: one Int,
```

```
171   var detectedProblems: set Problem,
172   start: one Timestamp,
173   end: one Timestamp
174 }
175
176 enum WeatherCondition {
177   Clear,
178   Partly_cloudy,
179   Cloudy,
180   Light_rain,
181   Heavy_rain,
182   Stormy,
183   Snowy,
184   Sunny,
185   Foggy,
186   Windy,
187   Not_defined
188 }
189
190 sig WeatherInfo {
191   forTrip: one Trip,
192   condition: one WeatherCondition,
193   temperature: one Int,
194   windSpeed: one Int,
195   when: one Timestamp
196 }
197
198 // Each query represents a request about a path, from every type of
      user
199 sig Query {
200   requester: one User,
201   origin: one Location,
202   destination: one Location,
203   results: set Path
204 }
205
206 ////---------- FACTS ------------
207
208 // Each trip has at most one weather recording
209 fact WeatherWellFormed {
210   all t: Trip | lone wi: WeatherInfo | wi.forTrip = t
211   all wi: WeatherInfo | wi.when = wi.forTrip.start
212   all wi: WeatherInfo |
213     wi.temperature >= -30 and wi.temperature <= 50 and
214     wi.windSpeed >= 0 and wi.windSpeed <= 200
215 }
216
217
218 // Ensures every problem has a reasonable severity
219 fact ProblemWellFormed {
220   all p: Problem |
221     p.severity >= 1 and p.severity <= 10
222 }
223
224
225 // Path endpoints must differ
226 fact PathEndpointsAreDistinct {
227   all p: Path | p.from != p.to
```

```
}


// The streets which compose a path are sequential, and not far located
fact PathContinuity {
  all p: Path | {
    some s: p.streets | s.begin = p.from
    some s: p.streets | s.end = p.to
    all s: p.streets | s.end != p.to implies
        one sNext: p.streets | sNext.begin = s.end
  }
}


// No negative street lengths
fact NoNegativeStreetLength {
  all s: Street | s.length >= 0
}

// Trip end after start
fact TripDurationWellFormed {
  all t: Trip | TSO/lt[t.start, t.end]
}

// Query endpoints distinct
fact QueryEndpointsAreDistinct {
  all q: Query | q.origin != q.destination
}

// Query results match endpoints
fact QueriesRespectEndpointsFact {
  all q: Query, p: q.results | p.from = q.origin and p.to = q.
     destination
}

// Small range for dates (avoid huge universe)
fact DateRanges {
  all d: Date |
    d.day >= 1 and d.day <= 31 and
    d.month >= 1 and d.month <= 12 and
    d.year >= 0 and d.year <= 7
}

fact TimeOfDayRanges {
  all t: TimeOfDay |
    t.hour   >= 0 and t.hour   <= 23 and
    t.minute >= 0 and t.minute <= 59 and
    t.second >= 0 and t.second <= 59
}

// Avoid timestamps with same date and time
fact NoDuplicateTimestamps {
  all disj t1, t2: Timestamp |
    not (t1.date = t2.date and t1.tod = t2.tod)
}


// Auto problems must be confirmed before publication
```

```
fact AutoProblemsNeedConfirmationForPublication {
  all ar: AutoRegistration |
    ar.visibility = Published implies
      all o: ar.problems | o.confirmation = Confirmed
}

// Lifecycle constraints for auto-detected problems (temporal)
fact AutoProblemLifecycle {
  all ar: AutoRegistration, o: ar.problems |
    //until isn't  Confirmed/Rejected, it has to be ToConfirm
    always (
      (o.confirmation != Confirmed and o.confirmation != Rejected)
        implies o.confirmation = ToConfirm
    )
    and
    //if Confirmed or Rejected, before there was ToConfirm
    always (
      (o.confirmation = Confirmed or o.confirmation = Rejected)
        implies once (o.confirmation = ToConfirm)
    )
    and
    always (
      (o.confirmation = Confirmed) implies after (o.confirmation =
          Confirmed)
    )
    and
    always (
      (o.confirmation = Rejected) implies after (o.confirmation =
          Rejected)
    )
}


// Speed constraints on AutoRegistration
fact SpeedConstraints {
  all ar: AutoRegistration |
    ar.minSpeed > 0 and
    ar.avgSpeed > 0 and
    ar.maxSpeed > 0 and
    ar.maxSpeed < 100 and
    ar.minSpeed < ar.maxSpeed and
    ar.minSpeed <= ar.avgSpeed and
    ar.avgSpeed <= ar.maxSpeed
}

// Exists a query done by a nonregistered user (scenario support)
fact SomeNonregisteredUserQueries {
  some q: Query | q.requester in NonregisteredUser
}

// Ensures that every street is well formed
fact StreetWellFormed {
  all s: Street |
    s.length > 0 and s.length <= 50 and
    s.begin != s.end
}


```

```
341 // Ensures that every trip is well formed
342 fact TripWellFormed {
343   all t: Trip |
344     TSO/lt[t.start, t.end] and
345     t.distance = t.path.length and
346     t.distance > 0
347 }
348
349
350 // No duplicate locations (same lat+long)
351 fact NoDuplicateLocations {
352   all disj l1, l2: Location |
353     l1.latitude != l2.latitude or l1.longitude != l2.longitude
354 }
355
356
357 // Clearing old problems, when the same path is marked as Optimal
358 fact OldActiveProblemsAreClearedAfterNewOptimalPublished {
359   all p: Path, pr: Problem |
360     always (
361       ( pr in p.activeProblems and
362         some r: Registration |
363           r.path = p and
364           r.visibility = Published and
365           r.status = Optimal and
366           TSO/lt[pr.time, r.time]
367       )
368       implies
369       after always (pr not in p.activeProblems)
370     )
371 }
372
373
374 // A cleared problem will never come back as active one
375 fact ClearedProblemsNeverComeBack {
376   all p: Path, pr: Problem |
377     always (
378       (historically (some r: Registration |
379         r.path = p and r.visibility = Published and r.status = Optimal
              and
380         TSO/lt[pr.time, r.time]
381       ))
382       implies
383       always (pr not in p.activeProblems)
384     )
385 }
386
387
388 // Ensures the aggregated status represents the most frequent
    registration status,
389 //defaulting to the most recent one in case of a tie.
390 fact AggregatedStatusMajorityWithRecency {
391   always all p: Path |
392     let rs = publishedRegistrations[p] |
393       (no rs implies no p.aggStatus) and
394       (some rs implies
395         some p.aggStatus and
396         let s = p.aggStatus |
```

```
            some { r: rs | r.status = s } and
            all s2: Status |
              (some { r: rs | r.status = s2 }) implies (
                # { r: rs | r.status = s } > # { r: rs | r.status = s2 }
                or
                (
                  # { r: rs | r.status = s } = # { r: rs | r.status = s2 }
                  and
                  not TSO/lt[ latestTimeForStatus[p, s], latestTimeForStatus[p, s2] ]
                )
              )
      )
}


// AggStatus exists when there is at least one rating
fact AggStatusDefinedWhenRatingsExist {
  all p: Path |
    (some publishedRegistrations[p]) implies some p.aggStatus
}


// We need to ensure consistency of path length
fact PathLengthConsistent {
  always all p: Path | p.length = pathLength[p]
}


// We need to ensure consistency of the score
fact PathScoreConsistent {
  always all p: Path | p.score = scoreComputation[p]
}


// Once a registration is published, it cannot become private again.
fact PublishFlagIsMonotone {
  always all r: Registration |
    (r.visibility = Published) implies after (r.visibility = Published)
}



//Problems cannot be removed from AutoRegistratio, but at most fromasctiveProblems
fact AutoRegistrationProblemsNeverDisappear {
  always all ar: AutoRegistration, pr: Problem |
    (pr in ar.problems) implies after (pr in ar.problems)
}


fact PathLengthBounded_0_100 {
  always all p: Path | p.length >= 0 and p.length <= 100
}

fact ActiveProblemsMustBeConfirmed {
  always all p: Path, pr: Problem |
```

```
    (pr in p.activeProblems) implies (pr.confirmation = Confirmed)
}



////---------- FUNCTIONS ------------

// Converting time of day in seconds
fun timeToSeconds[t: TimeOfDay]: one Int {
  t.hour.mul[3600].add[t.minute.mul[60]].add[t.second]
}

// Converting dates to an amount of days (approx, bounded universe)
fun dateToDays[d: Date]: one Int {
  d.year.mul[366].add[d.month.sub[1].mul[31]].add[d.day.sub[1]]
}

// Converting a timestamp into seconds
fun tsToSeconds(ts: Timestamp): one Int {
  dateToDays[ts.date].mul[24].mul[3600].add[timeToSeconds[ts.tod]]
}

// Weight associated to each path condition (Optimal > Medium >
    Sufficient > RequiresMaintenance)
fun statusWeight[s: lone Status]: one Int {
  (s = Optimal) => 5 else
  (s = Medium) => 4 else
  (s = Sufficient) => 3 else
  (s = RequiresMaintenance) => 2 else
  0
}


// Total length of the path (from streets)
fun pathLength[p: Path]: one Int {
  sum s: p.streets | s.length
}

// Score derived from aggStatus
fun qualityScore[p: Path]: one Int {
  some p.aggStatus implies statusWeight[p.aggStatus] else 0
}


// Final score of a path based on aggStatus and path length only
fun scoreComputation[p: Path]: one Int {
  let q   = statusWeight[p.aggStatus] |     // 0,2,4,7,10
  let pen = p.length.div[15] |               //each 15 km ~ -1 point
  let raw = q.sub[pen] |
  (raw < 0) => 0 else (raw > 5) => 5 else raw
}



// Best paths based on score
fun bestPaths[q: Query]: set Path {
```

```
  { p: q.results | all p2: q.results | scoreComputation[p] >=
      scoreComputation[p2] }
}

// All ratings which have been published by a registered user
fun publishedRegistrations[p: Path]: set Registration {
  { r: Registration | r.path = p and r.visibility = Published }
}

// Number of reports with state "Optimal" after a certain timestamp
fun optimalReportsAfter[p: Path, t: Timestamp]: set Registration {
  { r: publishedRegistrations[p] | r.status = Optimal and TSO/lt[t, r.
      time] }
}

// Most recent timestamp among published ratings of a given status
fun latestTimeForStatus[p: Path, st: Status]: lone Timestamp {
  { t: Timestamp |
    some r: publishedRegistrations[p] |
      r.status = st and r.time = t and
      all r2: publishedRegistrations[p] |
        r2.status = st implies not TSO/lt[t, r2.time]
  }
}

//// ------------------ ASSERTIONS -----------------

assert OnlyRegisteredHaveTrips {
  all t: Trip | t.owner in RegisteredUser
}

assert OnlyRegisteredRatePaths {
  all r: Registration | r.by in RegisteredUser
}

assert UnregisteredCanQuery {
  some q: Query | q.requester in NonregisteredUser
}

// If AutoRegistration is Published, all its problems are Confirmed
assert AutoPublishedProblemsConfirmed {
  all ar: AutoRegistration |
    ar.visibility = Published implies
      all o: ar.problems | o.confirmation = Confirmed
}

// Trip lasts no more than 24h and duration is positive
assert TripDurationAtMost24h {
  all t: Trip |
    let dur = tsToSeconds[t.end].sub[tsToSeconds[t.start]] |
      dur > 0 and dur <= 24.mul[3600]
}

assert BestPathsAreMaximal {
  all q: Query, p: bestPaths[q], p2: q.results |
    scoreComputation[p] >= scoreComputation[p2]
}

```

```
assert ShorterBetterGivenSameStatus {
  all p1, p2: Path |
    some p1.aggStatus and some p2.aggStatus and
    p1.aggStatus = p2.aggStatus and
    pathLength[p1] < pathLength[p2]
    implies scoreComputation[p1] > scoreComputation[p2]
}

assert AggregatedStatusRespectsMajorityAndRecency {
  all p: Path |
    some p.aggStatus implies
      let rs = publishedRegistrations[p] |
      some rs and
      let s = p.aggStatus |
        all s2: Status |
          # { r: rs | r.status = s } >= # { r: rs | r.status = s2 } and
          ( # { r: rs | r.status = s } = # { r: rs | r.status = s2 }
             and
            some { r: rs | r.status = s } and
            some { r: rs | r.status = s2 } )
          implies
            let ts  = latestTimeForStatus[p, s],
                ts2 = latestTimeForStatus[p, s2] |
              some ts2 implies (some ts and not TSO/lt[ts, ts2])
}

assert NoDuplicateTimestampsRespected {
  all disj t1, t2: Timestamp |
    not (t1.date = t2.date and t1.tod = t2.tod)
}

assert DateRangesRespected {
  all d: Date |
    d.day >= 1 and d.day <= 31 and
    d.month >= 1 and d.month <= 12 and
    d.year >= 0 and d.year <= 7
}

assert TimeOfDayRangesRespected {
  all t: TimeOfDay |
    t.hour   >= 0 and t.hour   <= 23 and
    t.minute >= 0 and t.minute <= 59 and
    t.second >= 0 and t.second <= 59
}

assert PathEndpointsDistinct {
  all p: Path | p.from != p.to
}

assert QueryEndpointsDistinct {
  all q: Query | q.origin != q.destination
}

assert QueriesRespectEndpoints {
  all q: Query, p: q.results |
    p.from = q.origin and p.to = q.destination
}
```

```
621 assert NoNegativeStreetLengthAssert {
622   all s: Street | s.length >= 0
623 }
624
625 assert ScoreIsNeverNegative {
626   always all p: Path | p.score >= 0
627 }
628
629 assert WeatherPerTripAtMostOneAssert {
630   all t: Trip | lone wi: WeatherInfo | wi.forTrip = t
631 }
632
633 assert WeatherTimeConsistentWithTripAssert {
634   all wi: WeatherInfo | wi.when = wi.forTrip.start
635 }
636
637 assert WeatherRangesAssert {
638   all wi: WeatherInfo |
639     wi.temperature >= -30 and wi.temperature <= 50 and
640     wi.windSpeed >= 0 and wi.windSpeed <= 200
641 }
642
643 assert AutoSpeedConstraintsAssert {
644   all ar: AutoRegistration |
645     ar.minSpeed > 0 and
646     ar.avgSpeed > 0 and
647     ar.maxSpeed > 0 and
648     ar.maxSpeed < 100 and
649     ar.minSpeed < ar.maxSpeed and
650     ar.minSpeed <= ar.avgSpeed and
651     ar.avgSpeed <= ar.maxSpeed
652 }
```

FIRST EXAMPLE: AutoRegistration confirmation and publication effects

This example shows how a problem reported through an AutoRegistration is first private and unconfirmed, then confirmed and published, and how the publication updates the path aggregated status (`aggStatus`).

In this Alloy scenario we analyze the same *AutoRegistration* across three consecutive states of the BBP system. In state 1, a registered user creates an *AutoRegistration* on a path: the registration has visibility *Private* and it reports a problem whose confirmation is *ToConfirm*. Since the registration is not published and the problem is not confirmed yet, the path does not expose an updated aggregated status.

In state 2, the reported problem becomes *Confirmed*, but the aggregated status is not yet modified, because the registration is still *Private*.

Finally, in state 3 the *AutoRegistration* is made *Public*. Thanks to this change, the path shows the new aggregated evaluation, because the `aggStatus` is a derived attribute that depends on what is currently published.

```
1  //In this scenario there's a view about the lifecycle of a problem,
       when it's detected and then published
2  pred ProblemLifeCycle {
3    some ar: AutoRegistration, p: Path, prob: Problem | {
4      ar.status = Medium
5      ar.path = p
6      prob in ar.problems
7
8      //The Porblem has just been created. It has to be validated
9      ar.visibility = Private
10     prob.confirmation = ToConfirm
11
12     //Validation
13     eventually {
14       prob.confirmation = Confirmed
15       ar.visibility = Private
16
17   //After validation, the aggStatus is modified
18       eventually {
19         prob.confirmation = Confirmed
20         ar.visibility = Published
21         p.aggStatus = ar.status
22       }
23     }
24   }
25 }
26
27 run ProblemLifeCycle  for 3 but
28   5 steps,
29   10 Int,
30   exactly 1 Path,
31   exactly 1 Street,
32   exactly 1 AutoRegistration,
33   exactly 1 Registration,
34   exactly 0 ManualRegistration,
35   exactly 1 Problem,
36   exactly 3 Timestamp,
37   exactly 1 Date,
38   exactly 3 TimeOfDay,
39   exactly 1 Query,
40   exactly 1 NonregisteredUser,
```

```
41    exactly 1 Trip,
42    exactly 0 WeatherInfo
```

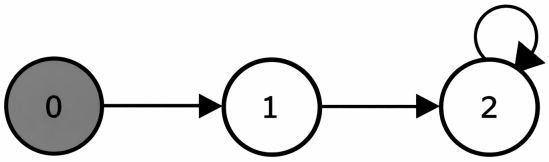


Figure 22: Scenario 1 - State Machine

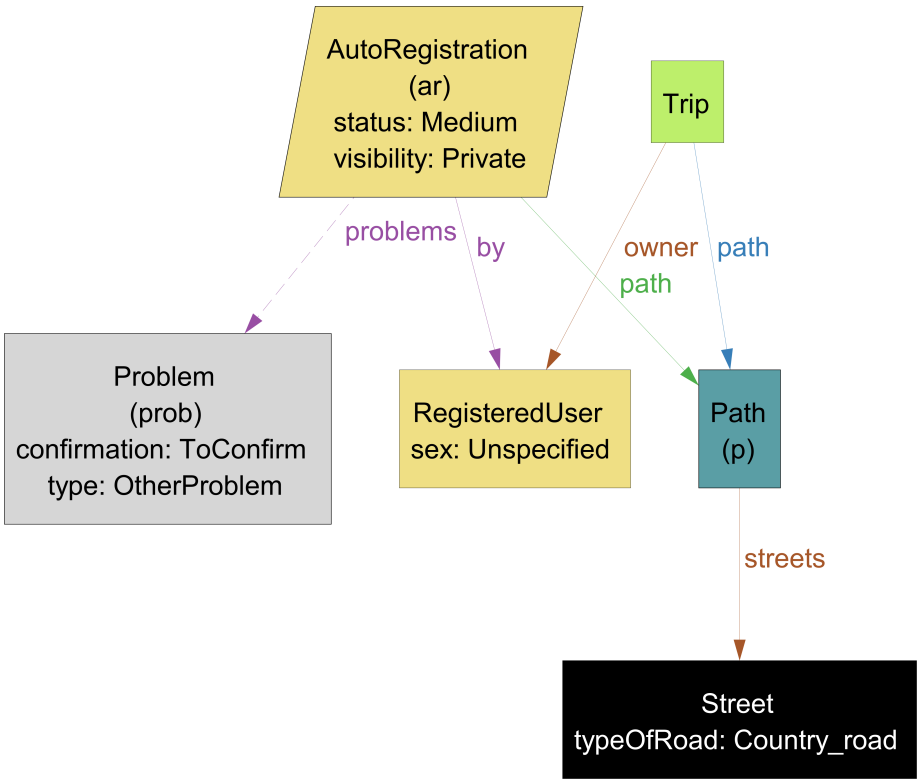


Figure 23: Scenario 1 - State 1

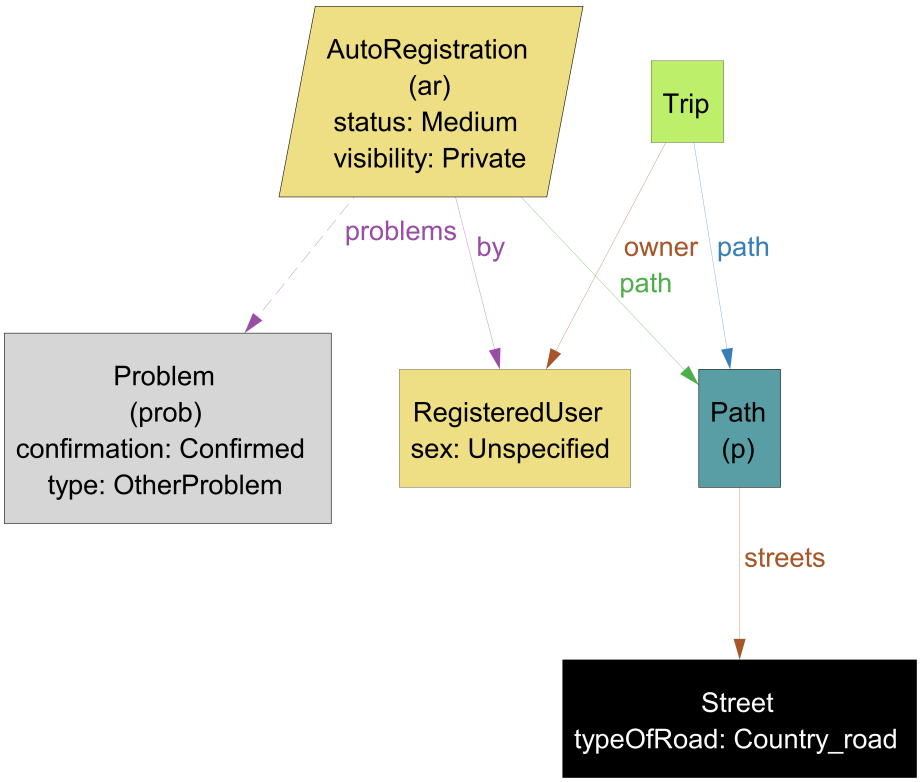


Figure 24: Scenario 1 - State 2

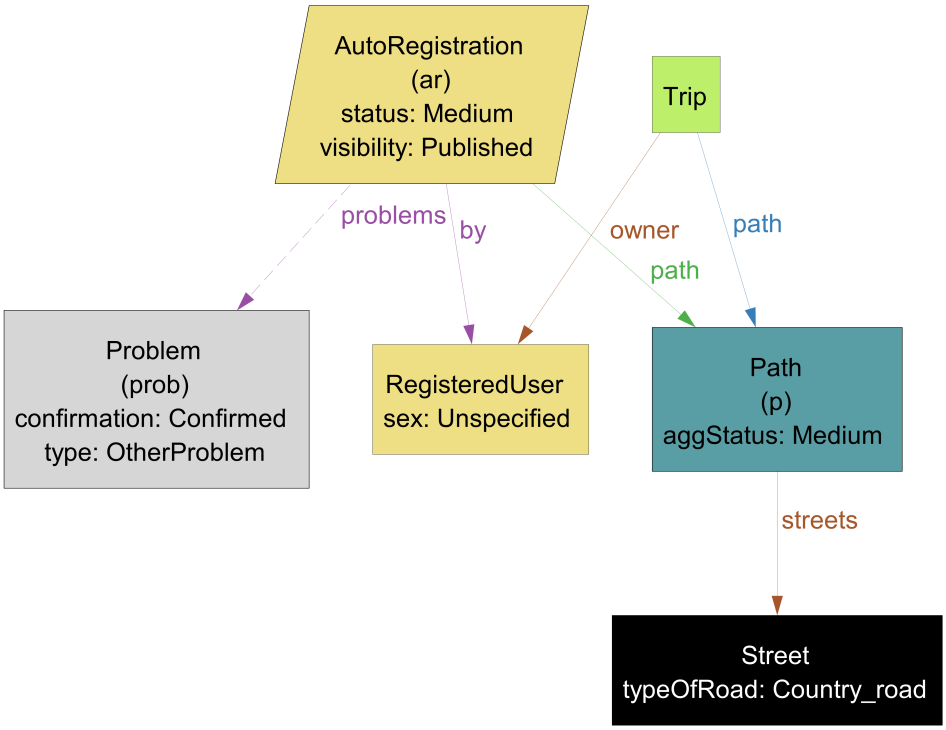


Figure 25: Scenario 1 - State 3

SECOND EXAMPLE: Manual Registrations and Aggregated Status Update, with two different Timestamps

This example illustrates how multiple registrations with different publication states and timestamps influence the aggregated status of a path.
In this Alloy scenario, we analyze the evolution of a path evaluation based on *manual registrations*. Initially (state 1), two manual registrations are associated with the same path: an older one with status *Sufficient* and a more recent one with status *Optimal*. Both registrations are marked as *private*, therefore none of them is visible to other users and the path's aggregated status (`aggStatus`) is not yet determined by these evaluations.
In the second state (state 2), the older manual registration with status *Sufficient* is published, while the more recent *Optimal* registration remains private. As a consequence, the path's aggregated status is updated to *Sufficient*, since it is the only published evaluation available at this point.
Finally, in the third state (state 3), the more recent manual registration with status *Optimal* is published as well. Given the aggregation policy that considers both publication status and temporal recency, this newer published evaluation overrides the previous one. As a result, the path's aggregated status is updated to *Optimal*.

```
1
2  //Here it's shown the case when the aggrStatus evolves due to the fix
       of problems
3  pred PathStatusEvolution {
4    some p: Path, s: Street, disj r1, r2: ManualRegistration, ru:
         RegisteredUser,
5          q: Query, nu: NonregisteredUser | {
6
7      p.streets = s
8      s.begin = p.from and s.end = p.to
9      s.length = 1
10
11     q.requester = nu
12     q.origin = p.from
13     q.destination = p.to
14     q.results = p
15
16     r1.path = p and r2.path = p
17     r1.by = ru and r2.by = ru
18     r1.status = Sufficient
19     r2.status = Optimal
20
21     //r2 comes after r1
22     let t0 = TSO/first, t1 = TSO/next[t0], t2 = TSO/next[t1] | {
23       all t: Timestamp | {
24         t.date.year = 0 and t.date.month = 1 and t.date.day = 1
25         t.tod.hour = 0 and t.tod.minute = 0
26       }
27       r1.time = t1
28       r2.time = t2
29     }
30
31     //No published registration --> path with no aggStatus
32     r1.visibility = Private
33     r2.visibility = Private
34     no publishedRegistrations[p]
35     no p.aggStatus
36
37     // First registration is published (Sufficient)
38     eventually {
39       r1.visibility = Published
```

```
        r2.visibility = Private
        p.aggStatus = Sufficient

        //  Second registration is published (Optimal)
        //r2 after r1 --> affStatus becomes Optimal
        eventually {
          r1.visibility = Published
          r2.visibility = Published
          p.aggStatus = Optimal
        }
      }
    }
}

run PathStatusEvolution for 5 but
  10 steps,
  10 Int,
  exactly 1 Path,
  exactly 1 Street,
  exactly 2 ManualRegistration,
  exactly 2 Registration,
  exactly 0 AutoRegistration,
  exactly 1 RegisteredUser,
  exactly 1 Query,
  exactly 1 NonregisteredUser,
  exactly 3 Timestamp,
  exactly 1 Date,
  exactly 3 TimeOfDay,
  exactly 0 Trip,
  exactly 0 WeatherInfo
```

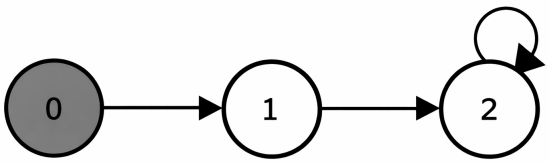


Figure 26: Scenario 2 - State Machine

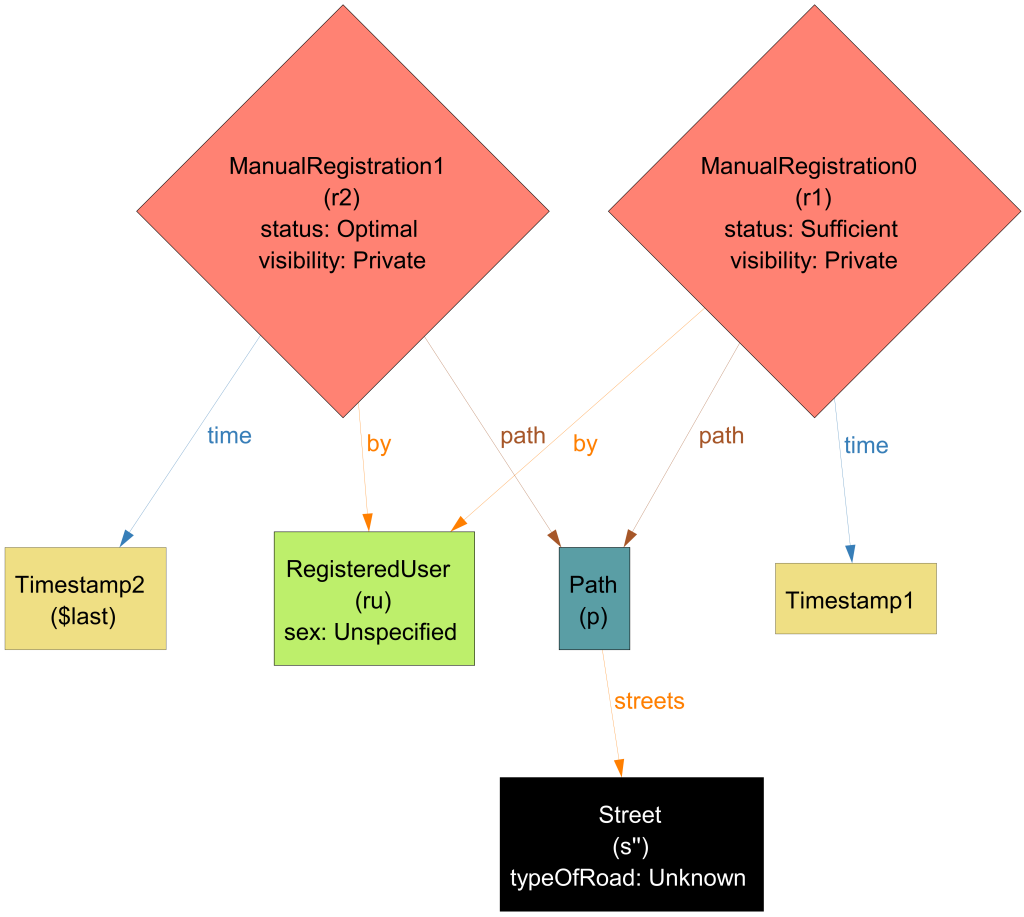


Figure 27: Scenario 2 - State 1

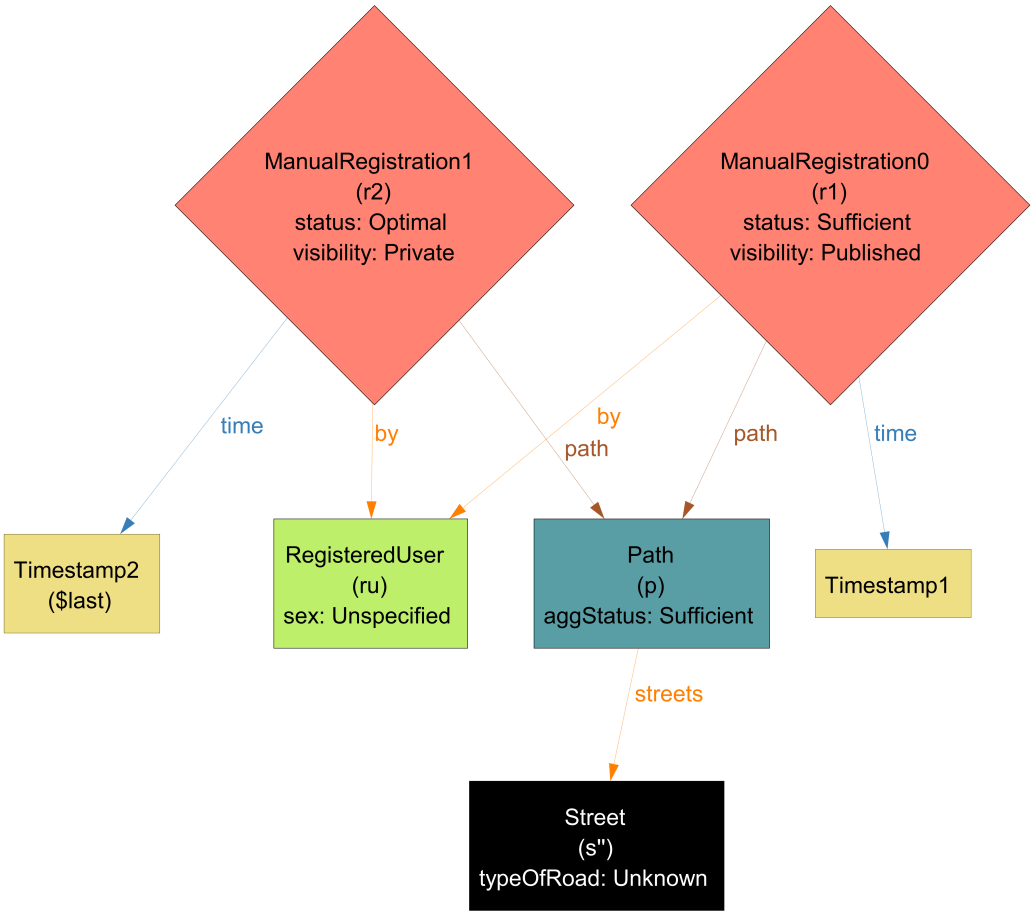


Figure 28: Scenario 2 - State 2

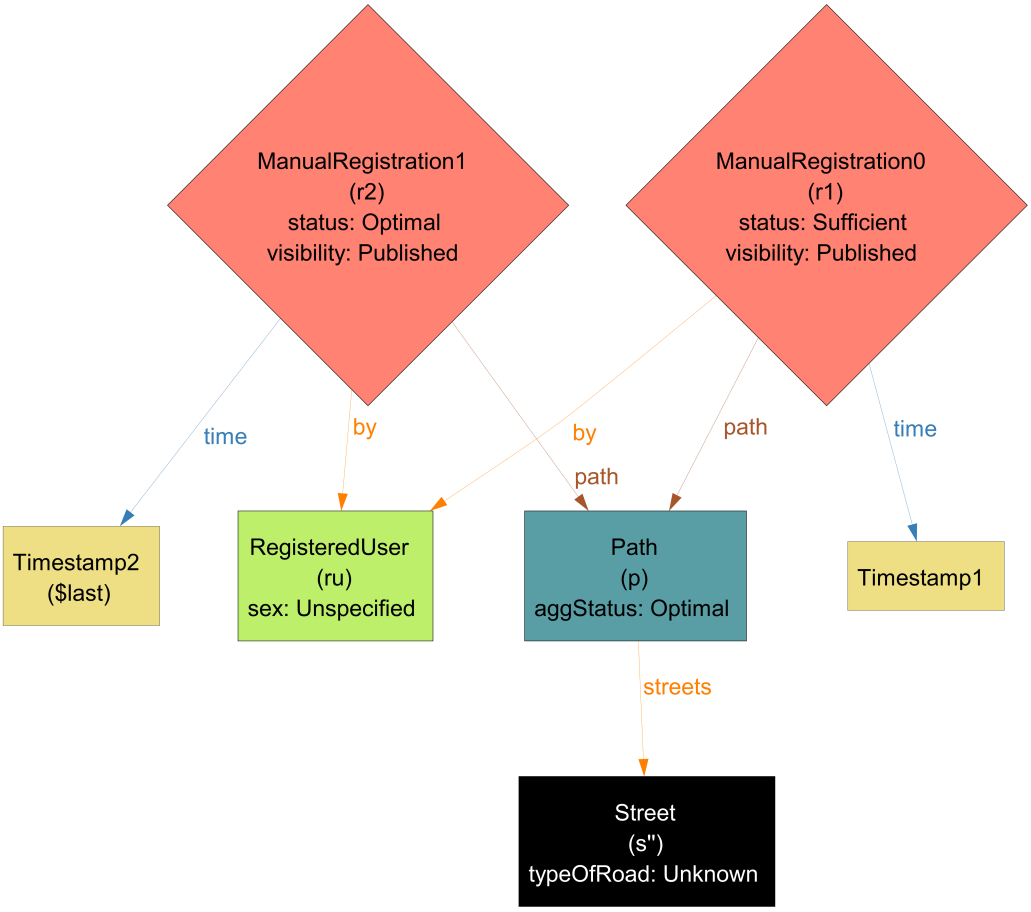


Figure 29: Scenario 2 - State 3

THIRD EXAMPLE: Impact of Registrations on aggStatus and Path Score

This example illustrates how the publication of registrations progressively affects both the aggregated status and the numerical score of a path.

In this Alloy scenario, we analyze a more articulated evolution in which the publication of registrations influences not only the aggregated status (`aggStatus`) of a path, but also its computed score. Initially (state 1), both a manual registration with status *Optimal* and an automatic registration with status *RequiresMaintenance* exist, but they are *private*. The automatic registration reports a problem whose confirmation status is *ToConfirm*. Since no registration is published at this stage, the path has no active problems, its aggregated status is not influenced by user feedback, and the resulting score is minimal (equal to 0).

In the second state (state 2), the automatic registration is published and the reported problem is confirmed. As a consequence, the problem becomes active for that path and starts influencing its computed score. The aggregated status of the path is updated to *RequiresMaintenance*, reflecting the only published evaluation available at this point. Accordingly, the path score increases to an intermediate value (e.g. 2), derived from the combination of the aggregated status and the structural characteristics of the path (such as its length).

Finally, in the third state (state 3), the manual registration with status *Optimal* is published. Since this registration is more recent and carries a better evaluation, it overrides the previous assessment according to the aggregation policy based on recency and quality. The path's aggregated status is therefore updated to *Optimal*, the previously active problem is no longer considered, and the score reaches its maximum value (e.g., 5).

```
1
2  //In this scenario the problem is fixed due to recency
3  pred FixingProblem {
4    some p: Path, s: Street, pr: Problem,
5         rOpt: ManualRegistration,
6         ar: AutoRegistration,
7         ru: RegisteredUser, q: Query, nu: NonregisteredUser | {
8
9      p.streets = s
10     s.begin = p.from and s.end = p.to
11     s.length = 1
12
13     q.requester = nu
14     q.origin = p.from
15     q.destination = p.to
16     q.results = p
17
18     ar.path = p
19     ar.by = ru
20     ar.status = RequiresMaintenance
21     pr in ar.problems
22
23     rOpt.path = p
24     rOpt.by = ru
25     rOpt.status = Optimal
26
27     let t0 = TSO/first, t1 = TSO/next[t0], t2 = TSO/next[t1] | {
28       all t: Timestamp | {
29         t.date.year = 0 and t.date.month = 1 and t.date.day = 1
30         t.tod.hour = 0 and t.tod.minute = 0
31       }
32       pr.time = t0
33       ar.time = t1
34       rOpt.time = t2
```

```
35      }
36
37      //The problem has to be confirmed (Private)
38      ar.visibility = Private
39      rOpt.visibility = Private
40      pr.confirmation = ToConfirm
41      pr not in p.activeProblems
42
43      // Publication auto registration and problem confirmed
44        eventually {
45        ar.visibility = Published
46        pr.confirmation = Confirmed
47        pr in p.activeProblems
48        p.aggStatus = RequiresMaintenance
49
50        // Manual registratioon published (Optimal), stronger due to
              recency
51        eventually {
52          rOpt.visibility = Published
53          p.aggStatus = Optimal
54
55         //Problem fixed
56          eventually {
57             pr not in p.activeProblems
58             rOpt.visibility = Published
59             p.aggStatus = Optimal
60          }
61        }
62      }
63    }
64  }
65
66  run FixingProblem for 5 but
67    10 steps,
68    10 Int,
69    exactly 1 Path,
70    exactly 1 Street,
71    exactly 1 Problem,
72    exactly 1 ManualRegistration,
73    exactly 1 AutoRegistration,
74    exactly 2 Registration,
75    exactly 1 RegisteredUser,
76    exactly 1 Query,
77    exactly 1 NonregisteredUser,
78    exactly 3 Timestamp,
79    exactly 1 Date,
80    exactly 3 TimeOfDay,
81    exactly 2 Location,
82    exactly 0 Address,
83    exactly 0 Trip,
84    exactly 0 WeatherInfo,
85    exactly 1 Text
```

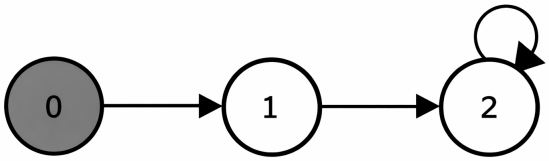


Figure 30: Scenario 3 - State Machine

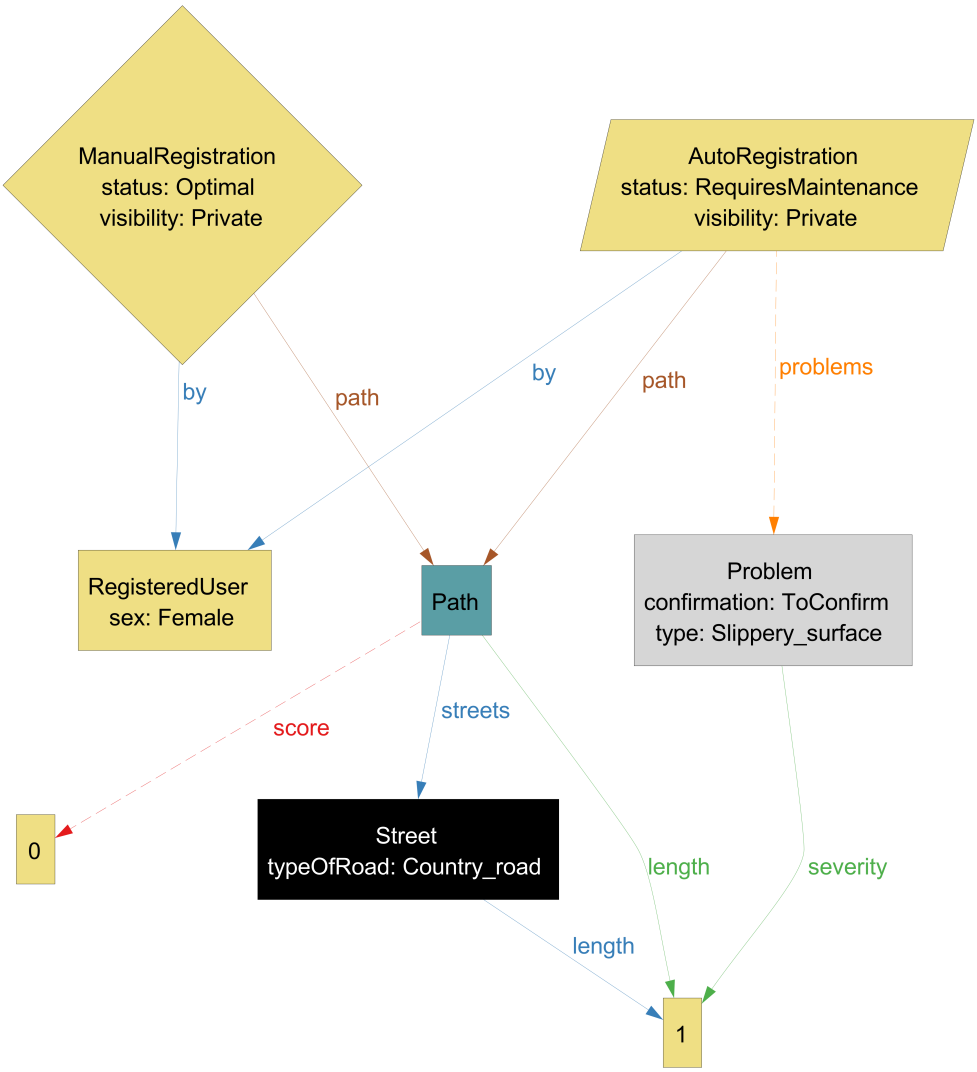


Figure 31: Scenario 3 - State 1

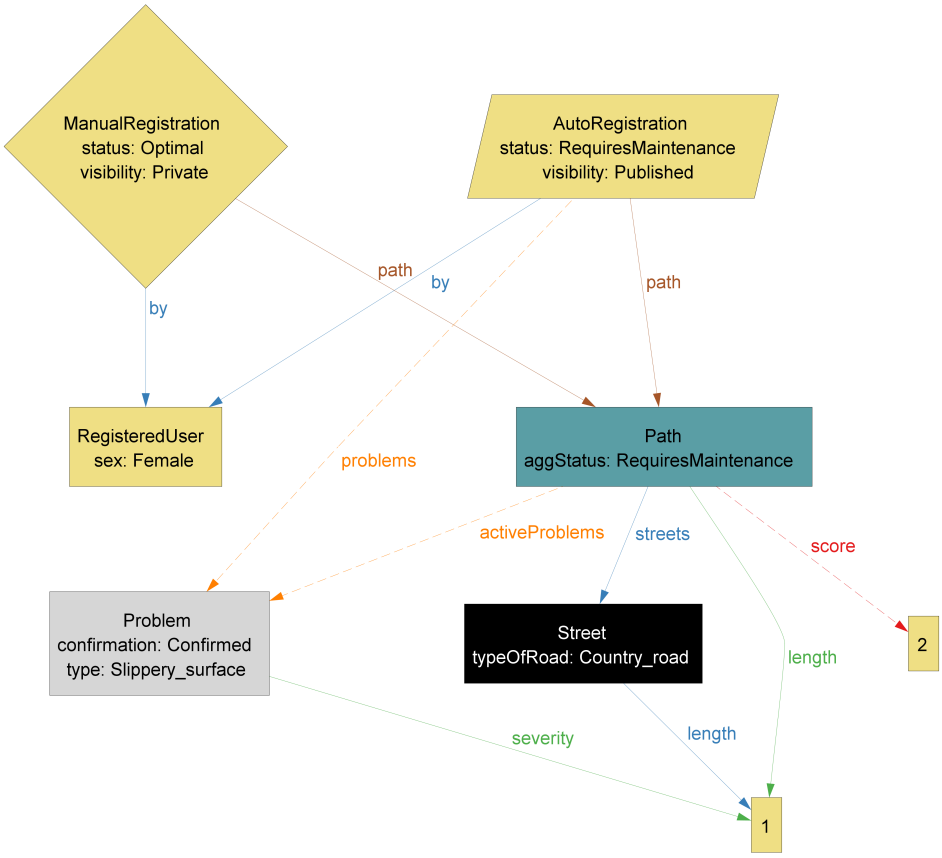


Figure 32: Scenario 3 - State 2

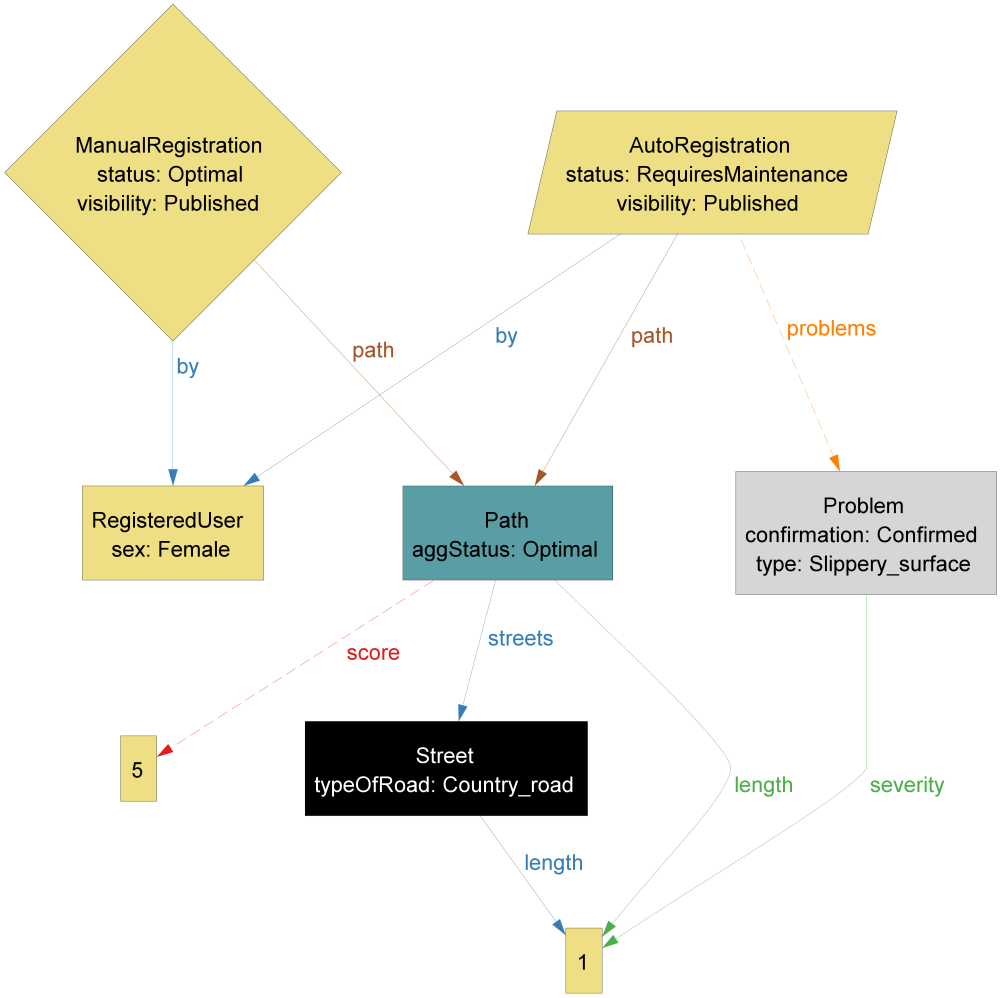


Figure 33: Scenario 3 - State 3

5 EFFORT SPENT

The following table summarizes the effort spent by each team member for the preparation of the main chapters of this document, expressed in working days.

Team Member	Chapter 1	Chapter 2	Chapter 3	Chapter 4
Andrea Rubagotti	3	4	5	4
Maria Concetta Santagata	3	4	5	4
Samuele Leanza	0	3	4	0

Table 13: Effort spent (in days) by each team member per document chapter

In addition, the README file included in the project repository provides a complete and detailed history of all document updates. This history shows all the revisions performed across the two different tools adopted during the project (Google Docs and Overleaf) and allows full traceability of contributions by date and author.

6 REFERENCES

The assignment for this document and all the information included refer to the following documentation:

References

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the WeBeep page of the Software Engineering II course [`https://webeep.polimi.it/`].

Used Tools

- **OpenAI** - *ChatGPT*. Used as an auxiliary tool for drafting and refining sections of this document.
- **GitHub** for project versioning and sharing.
- **LATEX and Overleaf** as editor for writing this document.
- **StarUML** for sequence diagrams.
- **draw.io** for other diagrams.
- **Figma** for the UI's design.
- **Google Documents** for collaborative writing, notes and reasoning.

Use of Generative AI

Generative AI tools were used in this project exclusively as support instruments and not as a replacement for the authors' reasoning or design decisions. In particular, ChatGPT was employed to provide targeted suggestions related to LaTeX usage, enabling us to better exploit its advanced features and to speed up repetitive or mechanical tasks, such as the creation and formatting of tables. The tool was also used to obtain confirmations and clarifications regarding pre-existing design ideas, which were always critically evaluated by us before being adopted. Moreover, ChatGPT was really useful in organizing the revision history of the project, because our development was carried out across both Google Docs and Overleaf (used sequentially during the project). The AI tool was used to produce a single well-structured list ready for inclusion in the documentation. All outputs generated by the tool were carefully reviewed, validated, and, when necessary, refined to ensure correctness, coherence, and consistency with the project's objectives and academic standards.